Book Buddy: A Multi-Tenant SaaS Platform for Smart Library

A Project Report

Submitted in partial fulfillment of the

Requirements for the award of the Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

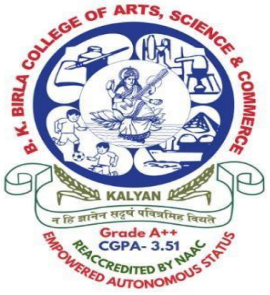
Pradnya Ravindra Pawar

5145015

Under the esteemed guidance of

Ms. Pritam Kamble

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE

B. K. BIRLA COLLEGE OF ARTS, SCIENCE & COMMERCE

(Affiliated to University of Mumbai)

KALYAN, 421301

MAHARASHTRA - 2026

B. K. BIRLA COLLEGE OF ARTS, SCIENCE & COMMERCE

(Affiliated to University of Mumbai)

KALYAN - MAHARASHTRA – 421301

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the project entitled, "**Book Buddy: A Multi-Tenant SaaS Platform for Smart Library and Lab Resource Management** ", is bonafied work of **PRADNYA RAVINDRA PAWAR** bearing Seat. No: **53261052** submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE in COMPUTER SCIENCE from University of Mumbai.

Internal Guide **Coordinator**

Date: **External Examiner** **College Seal**

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

(Note:All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)

PNR **No.: .....** Rollno: _____

1. Name of the Student _____
2. Title of the Project
   _____
3. Name of the Guide
   _____
4. Teaching experience of the Guide _____
5. Is this your first submission? Yes ☐ No ☐

Signature of the Student Signature of the Guide

Date: ..... Date: .....

Signature of the coordinator

Date: .....

Abstract

Academic libraries often rely on fragmented legacy systems to manage physical books, digital resources, computer laboratories, and student engagement. These disconnected processes cause operational inefficiencies, incomplete usage records, and limited accessibility. To address these challenges, this project presents **BookBuddy**, a production-grade, multi-tenant Integrated Library System (ILS) and Digital Learning Hub designed for universities, colleges, and academic consortia.

BookBuddy integrates physical library circulation, digital resource reading, facility reservation, and gamified learning into a single platform. It supports barcode and RFID-based book transactions, renewals, reservations, and overdue fine management. Its digital reader provides EPUB and PDF reading, highlights, notes, and offline access. The facility booking module manages laboratory workstations, no-shows, and waitlists, while gamification features encourage reading through streaks, badges, and leaderboards.

The platform uses the MERN-family stack, including React 19, Vite 8, Tailwind CSS v4, Zustand, and TanStack Query v5 for the frontend, with Node.js, Express 5, MongoDB, and Mongoose 9 for the backend. Redis supports session management and rate limiting, while Socket.io enables real-time communication. Multi-tenancy ensures institutional data isolation through college-level access controls, institution-specific subdomains, and configurable modules.

Key engineering features include JWT authentication with rotating refresh tokens, asynchronous bulk onboarding of more than 5,000 student records in batches of 500, live progress tracking, and Razorpay payment integration with HMAC-SHA256 verification. HTTP 206 range streaming enables efficient digital reading, while IndexedDB and Progressive Web Application (PWA) technologies support offline access. Background jobs automate fine accrual, reminders, hold expiry, payment reconciliation, and laboratory no-show handling.

Security and reliability are strengthened through CSRF protection, NoSQL injection prevention, SSRF defences, rate limiting, multi-factor authentication, automated CI quality gates, and backend integration testing. The application is deployed using Vercel and Render, with documented configurations for GCP-based deployment and recovery.

ACKNOWLEDGEMENT

I have put considerable effort into this project basically months of hard work. However, it would not have been possible without the kind support and assistance of many individuals. I would like to extend my sincere thanks to all of them.

I would like to express my heartfelt gratitude to my project guide **Ms. Pritam Kamble** for her continuous support, valuable insights, and guidance throughout the development of this project. Her encouragement and technical advice helped me stay focused and motivated at every stage.

I am also thankful to all the faculty members of the **Department of Computer Science, B. K. Birla College,** for their constant inspiration and for providing the resources necessary to complete this project.

A special thanks to my friends and family, who have always supported and believed in me, especially during the challenging stages of research and coding. Their encouragement helped me stay consistent and enthusiastic throughout the project journey.

Lastly, I want to thank myself for the dedication, effort, and patience that went into transforming an idea into a complete working system.

DECLARATION

I hereby declare that the project entitled, **“BookBuddy: Enterprise Multi-Tenant Campus Library,”** done at place where the project is done, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE (COMPUTER SCIENCE)** to be submitted as final semester project as part of our curriculum**.**

Chapter 1

Introduction

1.1 Background

Evolution of library systems. Library automation began with card catalogs and manual ledgers, and moved to computerised catalogs and circulation modules in the late twentieth century. Web-based OPACs followed, letting patrons search holdings remotely.Over time, ILS products grew to include acquisitions, serials management, and reporting. Many of these products were designed for a single institution, installed on that institution's own servers, and maintained by its own staff.

Limitations of the Traditional Model:

1. **Fragmentation:** Separate systems make library management difficult.
2. **High Cost:** Server maintenance and security require significant resources.
3. **Manual Facility Control:** Paper registers reduce booking efficiency.
4. **Limited Digital Reading:** Offline reading and annotation features are often unavailable.
5. **Low Engagement:** Lack of rewards reduces student participation.
6. **Manual Onboarding:** Student registration is time-consuming and error-prone.
7. **Payment Issues:** Cash payments create queues and accounting difficulties.

Multi-Tenancy Challenge: BookBuddy serves multiple colleges through a single cloud platform. It ensures data security through tenant isolation at the application and database levels, verified using automated security tests.

Technological Enablers

- **Single-Page Applications and REST APIs:** Provide a responsive user interface.
- **MongoDB:** Stores flexible library records efficiently.
- **Token-Based Authentication:** Secures user sessions across devices.
- **WebSockets:** Enable real-time updates for bookings and inventory.
- **Progressive Web Apps:** Support offline reading and access.
- **Payment Gateways:** Enable secure online fine payments.

Project Overview: BookBuddy modernizes library management using cloud technologies, secure APIs, automated testing, and real-time updates to provide an efficient and reliable platform for colleges.

1.2 Objectives

The primary objective of BookBuddy is to design, develop, test, and deploy a secure, scalable, and multi-tenant Integrated Library System for academic institutions. The system aims to integrate physical book circulation, digital resource management, facility reservations, and student engagement into a single web-based platform. It seeks to automate traditional library operations, improve accessibility, and provide real-time information to students and administrators. The platform also focuses on data security, operational efficiency, and a user-friendly experience for multiple educational institutions.

The **primary objectives** of Bookbuddy are as follows:

1. To Develop a Unified Library Management System
To integrate book circulation, cataloging, digital resources, facility booking, student management, and analytics into a single web platform.

2. To Ensure Security and Multi-Tenancy
To support multiple colleges on one platform with strict data isolation, secure authentication, JWT-based sessions, and institution-specific configurations.

3. To Automate Library Operations
To implement book search, barcode-based issue and return, renewals, reservations, student registration, bulk data imports, and library analytics.

4. To Provide Digital Reading and Real-Time Services
To enable online and offline reading, highlighting, note-taking, reading progress synchronization, and real-time updates for books, bookings, and inventory.

5. To Enhance Student Engagement and Accessibility
To introduce reading streaks, achievement badges, leaderboards, reviews, reading lists, seat reservations, waitlists, and secure online fine payments.

6. To Ensure Scalability, Reliability, and Maintainability
To provide administrator tools, audit logs, disaster recovery, automated testing, performance optimization, accessibility, and reliable deployment procedures.

7. To Simplify Student Registration and Administration
To enable efficient student onboarding through bulk data imports, automated validation, progress tracking, and error reporting, reducing manual effort and registration errors.

8. To Improve Library Management and Decision-Making
To provide reports and analytics on book circulation, resource utilization, student reading activities, reservations, and fines, helping administrators make informed decisions.

1.3 Purpose, Scope and Applicability

1.3.1 Purpose

The purpose of BookBuddy is to give academic institutions a single, affordable, and modern platform for managing library resources and engaging students, without the infrastructure burden of traditional systems. The purpose differs somewhat for each group of users:

Students and patrons gain one place to search physical and digital holdings, read e-books with their own annotations, reserve lab seats, renew loans, pay fines, and track their reading progress. The aim is to remove the friction that discourages library use. • Librarians and college administrators gain tools that remove manual work from circulation, inventory, finance, and onboarding, and that give them live visibility into library activity. They keep control over which features their students see.

Institutions and consortia gain a shared platform that supports inter-library sharing and cross-college catalogue search, with each college's data kept separate. Platform operators gain the governance, monitoring, and recovery tools needed to run a multi-tenant service safely.

The project has a second, academic purpose. It serves as a complete case study in applying software engineering principles to a real-world problem. These include requirements analysis, layered architecture, secure design, concurrency control, performance

optimisation, automated verification, and production deployment. The report is intended to show that these principles can be combined in a system that is functionally rich and also operationally sound.

1.3.2 Scope

The scope of BookBuddy extends beyond traditional library management. It integrates physical book circulation, digital reading, facility reservations, and student engagement into a single web-based platform. The system aims to simplify library operations, improve resource accessibility, and provide an efficient experience for students and administrators across multiple educational institutions.

The application covers four main components:

1. **Library Management** – Managing book catalogs, searching, issuing, returning, renewals, inventory, and student records.
2. **Digital Library** – Providing access to digital resources, including EPUB and PDF reading, annotations, reading progress synchronization, and offline access.
3. **Facility and Student Engagement** – Managing computer-lab and study-seat reservations while encouraging reading through badges, streaks, leaderboards, and reviews.
4. **Administration and Analytics** – Supporting multi-college management, secure access, online fine payments, real-time updates, reports, and library usage monitoring.

Technologically, BookBuddy uses modern web technologies, database management, secure authentication, and real-time communication to provide a responsive and scalable user experience. The platform supports multiple institutions with isolated data and customizable features, making library services more accessible, secure, and efficient.

1.3.3 Applicability

BookBuddy is applicable to colleges, universities, schools, research centres, and organisations that require efficient library and resource management. It simplifies book management, digital resource access, and facility booking through a centralised platform.

1. Colleges and Universities: Manages books, student records, and library transactions.
2. Academic Libraries: Organises book catalogues and supports inter-library resource sharing.
3. Schools and Training Institutes: Simplifies library operations and resource tracking.
4. Research and Corporate Libraries: Manages documents, digital collections, and facility reservations.

BookBuddy uses modern web technologies, cloud hosting, and database management to reduce manual work, improve accessibility, and provide efficient resource management**.**

1.4 Achievements

The development of BookBuddy has resulted in several significant technical and functional achievements. The project successfully integrates library management, digital resource access, facility booking, and administrative operations into a centralised platform. It provides role-specific portals for students, library staff, college administrators, and super administrators, simplifying daily activities and improving resource management.

One of the major achievements of BookBuddy is its comprehensive data management system, which uses 73 Mongoose models and more than 120 REST API endpoints to manage books, users, transactions, and institutional records. Its multi-tenant architecture allows multiple colleges to operate independently within a shared platform while maintaining data security and separation.

Another important achievement is the implementation of secure authentication with refresh-token rotation, token revocation, and synchronised logout functionality. These features help protect user accounts and improve session management. The system also supports real-time communication through Socket.IO, enabling timely updates across relevant dashboards.

BookBuddy also provides bulk student registration, allowing more than 5,000 student records to be imported in batches with progress tracking and downloadable error reports. This functionality reduces manual data entry and simplifies the onboarding process for educational institutions.

From a library management perspective, the system supports book cataloguing, digital reading, inter-library resource sharing, and facility reservations. Digital reading features include position synchronisation, annotations, and offline access. The facility booking module incorporates concurrency-safe reservations, automatic release of no-show bookings, and waitlist promotion, improving the utilisation of shared resources.

Performance optimisation has further improved the system's efficiency. Registration latency was reduced from approximately 1,200 milliseconds to 150 milliseconds per request. Additionally, frontend code splitting reduced the primary entry bundle from over 1.8 MB to cacheable chunks smaller than 350 KB, improving application loading performance.

The project also emphasises software quality through automated verification, security checks, and testing. Ten continuous integration quality gates help identify issues related to security vulnerabilities, data isolation, database queries, transaction handling, and concurrency. The testing process includes more than 80 backend integration test suites, frontend testing, and load testing using k6 and Artillery.

Overall, BookBuddy demonstrates the successful application of modern web technologies, database management, security practices, and performance optimisation to library automation. Its modular architecture, real-time capabilities, and scalable design make it a flexible solution for educational institutions. The project also provides valuable practical experience in software engineering, API development, database design, authentication, testing, and deployment while establishing a foundation for future enhancements.

1.5 Organisation of Report

This report is systematically organised into seven chapters to provide a clear and comprehensive understanding of the BookBuddy project. Each chapter explains a specific phase of the project, from its introduction and technology analysis to system design, implementation, testing, and future improvements.

Chapter 1: Introduction: presents the background, objectives, purpose, scope, and applicability of BookBuddy. It also highlights the major achievements and explains the organisation of the report.

Chapter 2: System Analysis: discusses the technologies, tools, frameworks, and database systems used in developing BookBuddy. It examines relevant existing systems and explains the selection of suitable technologies for the project.

Chapter 3: System Design: explains the overall system architecture, modules, database design, data flow diagrams, algorithms, user interface, and security mechanisms used in BookBuddy.

Chapter 4: Implementation and Testing: describes the implementation of the project's major modules, programming technologies, API integration, and testing methods. It covers unit testing, integration testing, and system performance evaluation.

Chapter 5: Results and Discussion: presents the system's outputs, interface screenshots, testing results, and performance observations. It discusses how BookBuddy fulfils its objectives and improves library and resource management.

Chapter 6: Conclusion and Future Scope: summarises the project's achievements, significance, and limitations. It also discusses possible future enhancements, including improved digital library services, additional integrations, and expanded resource management capabilities.

Chapter 7: References

Overall, this report provides a structured explanation of Book Buddy’s development, demonstrating how modern technologies can be used to create an efficient, secure, and scalable library management system.

Chapter 2

Survey of Technologies

2.1 Existing Systems

The systems currently used by academic libraries fall into three broad categories.

1. Manual and paper-based processes: Smaller libraries rely on accession registers, issue ledgers, and index cards. Computer-lab access is recorded in a paper register in which the student writes a name, time, and system number. Entries are often incomplete, usage cannot be searched or totalled, and nothing enforces the time limit. Students frequently remain at a seat after their permitted hour.

2. Standalone or on-premise library software: Many institutions use established open source or commercial ILS products that are installed and hosted locally. These handle the core catalog and circulation functions well, but they are typically designed for a single institution. Upgrades, backups, security patches, and server maintenance fall on the institution's own staff.

3. Disconnected supporting tools: Digital resources are kept in a separate repository or in shared folders. Rosters are managed in spreadsheets. Fines are collected in cash. Facility bookings are handled in a register or an unrelated booking tool. Student engagement is not tracked at all. Limitations of the existing system.

Limitations of the existing system.

Area	Limitation
Integration	Physical, digital, and facility services live in separate silos with no common view or shared identity.
Hosting and cost	On-premise installs need servers and technical staff, which smaller colleges cannot sustain.

Multi-institution support	Systems are single-institution by design, so consortia cannot share holdings or run one platform.
Digital reading	Resources are plain downloads, with no in-browser reader, annotations, position sync, or offline mode.
Facility control	Paper registers cause incomplete records, no analytics, no real-time availability, and no automatic release of unused seats.
Onboarding	Student rosters are entered manually or through slow imports that can block the system.
Payments	Fines are collected at the counter only, creating queues and manual reconciliation.
Real-time visibility	Stock and loan changes appear only after a refresh, so staff and patrons see stale data.
Engagement	No reading streaks, badges, reviews, or community features to encourage use.
Security	Many older systems lack token rotation, tenantlevel isolation, audit trails, and automated security checks.
Access	Interfaces are often desktop-only, not responsive, not accessible, and available in one language.

2.2 Problem Definition

Academic institutions need a single, affordable, secure system that manages physical circulation, digital resource access, facility reservation, and student engagement for one or many colleges. At present, these functions are handled separately through paper registers, on-premise software, and spreadsheets. The result is operational inefficiency, incomplete records, poor student experience, and high infrastructure cost.

Core problems identified.

1. **Fragmentation of services.** Students and staff must move between several tools to complete related tasks, and no one has a complete view of library activity.
2. **Absence of reliable facility control.** Lab and study seats are granted on trust, and paper registers do not prove that a student actually occupied a seat or left on time.
3. **High cost and maintenance burden** of locally hosted systems, which excludes smaller institutions from modern library services.
4. **Lack of safe multi-tenancy.** A shared platform for many colleges risks exposing one college's data to another unless isolation is designed in and verified.
5. **Weak digital learning support.** Students cannot annotate, resume, or read offline, which limits the value of the digital collection.
6. **Slow and error-prone onboarding.** Importing thousands of student records synchronously causes timeouts and blocks other users.
7. **Manual financial handling** of fines, with no digital payment or automated reconciliation.
8. **Low engagement** and no measure of how students use the library over time.

Problem boundaries. The problem addressed here is the design and delivery of a software platform. It does not include procurement of library stock, building of physical lab infrastructure, or institutional policy decisions such as loan periods. These are configurable inputs to the system, not outputs of it.

Success criteria. The solution is considered successful if a college can onboard its students in bulk, issue and return books by barcode, let students read and annotate ebooks, book lab seats without conflicts, collect fines online, and see live reports, all without any other tenant being able to view its data.

2.3 Requirements Specification

2.3.1 User Classes

User class	Description	Access level
Visitor (General)	Unauthenticated or nontenant user	Public catalog search, e-resource previews, local bookmarks
Student	Enrolled patron of one college	Own loans, holds, fines, reader, bookings, engagement features
College Admin	Library staff for one college	All thirteen desks, scoped to that college
Super Admin	Platform operator	Cross-tenant governance, with MFA for sensitive actions
Hardware Scanner	Gate kiosk or turnstile	Patron verification API only, using a secret key

2.3.2 Functional Requirements

ID	Module	Requirement
FR-01	Authentication	Users shall register, log in, and log out. The system shall issue a 15-minute access token and a rotating refresh token, and shall revoke all sessions if a refresh token is reused.
FR-02	Tenancy	The system shall resolve a college from its subdomain or path and shall scope every query to that college.
FR-03	Roles	The system shall enforce Student, College Admin, and Super Admin permissions on every protected route.

FR-04	Catalog	Staff shall add and edit titles with ISBN auto-fetch, shelf address, and barcode copies. Patrons shall search with facets and see live copy counts.
FR-05	Circulation	Staff shall check out and return items by barcode, enforce renewal limits, mark items lost, and fulfil holds in queue order.
FR-06	Holds and alerts	Students shall reserve unavailable titles, see queue position, and subscribe to availability alerts.

FR-07	Fines	The system shall calculate overdue fines daily up to a configured maximum, and staff shall record cash payments and waivers with a justification note.
FR-08	Payments	Students shall pay fines through Razorpay, and the system shall verify signatures and process webhooks idempotently.
FR-09	E-resources	Admins shall upload and moderate EPUB and PDF resources. Users shall read them in the browser with range streaming.
FR-10	Annotations	Students shall create, view, and delete highlights and notes, and shall resume reading at the saved position on any device.
FR-11	Offline reading	Students shall download resources for offline reading and view their annotations without a network.
FR-12	Lab booking	Students shall book workstations and seats within weekly caps. The system shall prevent doublebooking, release no-shows after the grace period, and promote waitlisted students.
FR-13	Facility setup	Admins shall define labs, bulk-generate seats, tag hardware specifications, and toggle maintenance mode.

FR-14	Bulk onboarding	Admins shall upload a CSV roster. The system shall process it asynchronously in chunks, show live progress, and return an error report.
FR-15	Engagement	The system shall provide daily check-ins, streaks with freeze tokens, badges, leaderboards, reviews, reading lists, and a community feed.
FR-16	Inter-library loan	Students shall search partner catalogs and track share requests. Admins shall approve, ship, and confirm returns.
FR-17	Acquisitions	Admins shall manage purchase orders, vendor quotes, receiving batches, and budgets.
FR-18	Feature management	Admins shall switch modules on or off, and the interface shall hide disabled modules and block direct access.
FR-19	Real-time	The system shall push circulation, inventory, patron, and upload-progress events to the right rooms and update client caches.
FR-20	Notifications	The system shall send in-app, email, and SMS notices for due dates, holds, and streak reminders.
FR-21	Reporting	Admins shall view circulation trends and top titles, and export CSV reports.
FR-22	Platform governance	Super admins shall provision tenants, review onboarding applications, moderate content, search audit logs, and impersonate users with MFA and a visible banner.
FR-23	Patron card and gate	Students shall show a digital card with a barcode and QR code, and the scanner API shall verify it.
FR-24	Disaster recovery	Super admins shall run restore only with a confirmation phrase, maintenance mode on, and MFA step-up.

2.3.3 Non-Functional Requirements

Category	Requirement
Performance	Landing page Largest Contentful Paint under 2.5 s, Cumulative Layout Shift under 0.1, Interaction to Next Paint under 200 ms. Main bundle split into chunks under 350 KB. Scanner verification latency under 50 ms.
Scalability	Support multiple tenants and multiple server instances, with Redis-backed sockets and rate limits. Handle rosters of 5,000 or more records per upload.
Security	Hashed passwords, CSRF protection, input sanitisation, rate limiting, Helmet headers, SSRF allowlists, immutable audit logs, and no secrets in the client bundle.
Data isolation	No query may run without tenant scoping, and compound indexes shall be prefixed by the tenant identifier.
Consistency	Contended updates (seats, loan counters, fines) shall use atomic conditional updates and transactions.
Availability	Health and version endpoints, scheduled jobs with failure isolation, and a documented rollback procedure.
Usability	Single-screen layouts with internal scrolling, onboarding tour, and clear feedback on every action.
Accessibility	WCAG 2.1 AA targets, keyboard navigation, focus rings, and colour contrast of at least 4.5:1.
Responsiveness	Supported from 320 px to 1920 px, with touch targets of at least 44 by 44 px.

Internationalization	English, Hindi, Marathi, and Spanish, with persistence of the chosen language.
Maintainability	Shared contract package, canonical role constants, ten automated CI gates, and written architecture documents.
Reliability	Idempotent payment webhooks, cron locks against overlap, and tested concurrency invariants.

2.3.4 Constraints and Assumptions

1. The system runs on cost-conscious infrastructure, including a 1 GB RAM server and a free-tier database, so heavy processing such as OCR is kept off the server.
2. Students have access to a modern web browser. Staff have access to barcode scanners if they want hardware-based circulation.
3. Institutions can provide student rosters as CSV files. Third-party services (payments, SMS, email, search, image hosting, and external catalogs) are available and are accessed over HTTPS.
4. Internet Connectivity**:** The system requires a stable internet connection for real-time updates, online payments, notifications, and synchronization of library records. Offline reading is available for supported digital resources.
5. Data Accuracy and User Responsibility**:** Institutions are responsible for providing accurate student records, book catalogues, and facility information. Users must follow the library's borrowing, payment, and booking policies.

2.4 Planning and Scheduling

2.4.1 Planning

Development approach. The project followed an iterative and incremental, Agile-inspired approach. Product requirement documents and implementation plans defined each major dashboard before it was built. Work was delivered in vertical slices (data model, API, interface, tests), and each slice was verified before the next began. Architecture and

contributor guidelines were written down so that design rules stayed consistent across the codebase.

Work breakdown. The work was divided into the following packages:

1. Requirements, feasibility, and architecture
2. Tenancy, authentication, and role-based access
3. Core ILS modules (catalogue, circulation, patrons, fines, acquisitions)
4. Role portals (General, Student, College Admin, Super Admin)

Risk	**Impact**	**Mitigation**
Cross-tenant data leakage	Critical privacy breach	Mandatory tenant filter, college ID -prefixed indexes, static tenant-isolation audit in CI
Race conditions on seats, loans, and fines	Double-booking or wrong balances	Atomic conditional updates, transactions, concurrency test suite
Refresh-token theft	Account takeover	Token rotation, family revocation, cross-tab logout
Slow bulk imports	Timeouts and blocked server	HTTP 202 asynchronous pipeline with chunked inserts
Free-tier resource limits	Outages or slow queries	Bundle budgets, index checks, bounded aggregations, cron jobs run off the request path
Third-party API failure	Broken search, payments, or alerts	Caching, timeouts, fallbacks, reconciliation jobs
Scope creep	Delay and instability	Roadmap tiers (build now, build with modification, defer)
Regression as features grow	Hidden defects	Ten CI gates, integration and landing-page regression suites

1. Digital reader, annotations, and offline support
2. Facility booking and gamification

3. Payments, bulk ingestion, real-time layer, and background workers
4. Security hardening, shared contracts, and CI quality gates
5. Testing, load testing, and performance tuning
6. Deployment, documentation, and runbooks **Risk analysis.**

Resources. The project needs development workstations, a version control and CI service, cloud hosting for client, API, database, and cache, and third-party service accounts for payments, SMS, email, image hosting, and search. Section 3.7 and Section 3.8 list the detailed requirements.

2.4.2 Scheduling

The schedule below is an indicative 24-week plan with overlapping phases.

Phase	**Activities**	**Weeks**	**Deliverable**
1. Requirements and feasibility	Observation, stakeholder analysis, requirement list, feasibility study	1–2	Requirement specification
2. Architecture and design	Component topology, tenancy model, data model, API contracts	3–5	Architecture and schema design
3. Foundation	Tenant resolution, authentication, token rotation, role-based access	5–8	Secure multi-tenant base
4. Core ILS	Catalog, circulation, patrons, holds, fines, acquisitions	7–11	Working library back office
5. Role portals	General, Student, College Admin, and Super Admin interfaces	9–15	Four portals

6. Digital reader	EPUB and PDF reader, annotations, position sync, offline mode	12–15	Reader module
7. Facilities and engagement	Lab grid, no-show release, waitlist, streaks, badges, leaderboards	14–17	Booking and gamification
8. Payments, ingestion, real-time	Razor pay, bulk CSV pipeline, Socket.io rooms, cron workers	16–19	Automation layer
9. Security and quality gates	CSRF, sanitization, rate limits, shared contracts, ten CI gates	18–21	Hardened platform
10. Testing and tuning	Integration, concurrency, load tests, bundle and index optimization	20–23	Test reports
11. Deployment and documentation	Vercel and Render deployment, smoke tests, runbooks, report	23–24	Live system and documents

Milestones. M1 is the approved requirement specification (week 2). M2 is the secure multi-tenant base (week 8). M3 is the working library back office (week 11). M4 is feature complete (week 19). M5 is the hardened and fully tested release (week 23). M6 is the production deployment (week 24).

2.5 Hardware Requirements

Development environment:

Component	Minimum	Recommended
Processor	Dual-core, 2.0 GHz	Quad-core or better, 2.5 GHz or higher
Memory	8 GB	16 GB (to run database, cache, API, and frontend together)

Storage	20 GB free	50 GB free SSD
Network	Broadband internet	Stable broadband for package installs and cloud services
Display	1366 by 768	1920 by 1080 or larger

Production server side:

Component	Specification
API host	Cloud web service (Render), or a GCP Compute Engine VM with at least 1 GB RAM and 1 vCPU. A second instance can be added behind a load balancer.
Database	MongoDB Atlas replica set (a free or shared tier is sufficient for pilots).
Cache and sessions	Managed Redis (for example Upstash or Redis Cloud).
Client hosting	Vercel global edge network.
Asset delivery	Cloudinary CDN for covers and e-resources, with Cloudflare in front for TLS and caching.

Client side:

Component	**Specification**
Device	Any laptop, desktop, tablet, or smartphone with a modern browser
Memory	2 GB or more for smooth e-book reading
Screen	320 px width or larger
Storage	Free space for offline downloads, if the student uses them

Network	Internet connection, except when reading downloaded resources

Optional library peripherals:

- Barcode scanners (USB or Bluetooth) for circulation and inventory.
- RFID readers and turnstile or kiosk hardware that can call the secured scanner verification endpoint.
- A label printer for barcodes and printed activation handouts.
- A lab-assistant tablet or phone for QR check-in and manual override.

2.6 Software Requirements

Software	**Version or choice**	**Purpose**
Operating system	Windows, macOS, or Linux for development. Linux for production.	Host environment
Node.js	v22 LTS or later (v24 in production)	Backend runtime and frontend build
npm	v10 or later	Package management with workspaces
MongoDB	v6.0 or later	Primary database
Redis	v7.0 or later	Sessions, rate limiting, socket adapter, feature cache
Git and GitHub	Current	Version control and CI workflows

Web server	Nginx (for GCP deployment)	Reverse proxy and SPA fallback

Backend stack:

Library	**Role**
Express 5	REST API framework
Mongoose 9	Object modelling for MongoDB
Socket.io 4 with Redis adapter	Real-time communication
Zod	Environment and request validation, shared contracts
Argon2id and bcrypt	Password hashing
node-cron	Scheduled background jobs
rate-limiter-flexible	Tiered rate limiting
Helmet, express-mongo-sanitize, hpp	Security headers and input sanitisation
csv-parser	Streaming CSV ingestion
Nodemailer and Twilio SDK	Email and SMS delivery
Passport.js	Optional Google and GitHub sign-in

Frontend stack.

Library	**Role**
React 19 and Vite 8	Single-page application and build tooling
TailwindCSS v4	Styling
Zustand 5	Client state
TanStack Query v5	Server-state caching

React Router	Routing
Epub.js and pdfjs-dist	EPUB and PDF rendering
idb and service workers	Offline storage and PWA
Recharts and Framer Motion	Charts and animation
dnd-kit	Drag-and-drop shelves
i18next	Internationalisation

Testing and quality tools. Jest for backend integration tests, Vitest with React Testing Library for the frontend, k6 and Artillery for load testing, ESLint, and custom AST scripts for the ten CI gates, run through GitHub Actions.

External services.

Service	**Purpose**
Razorpay	Fine payments and webhooks
Cloudinary and Vercel Blob	Cover and e-resource storage
Algolia	Fast full-text search
Google Books, Open Library, Project Gutenberg (Gutendex)	External catalog harvesting
Twilio and SMTP provider	SMS and email notices
Sentry	Error monitoring
Vercel, Render, Cloudflare, MongoDB Atlas	Hosting, edge, and database

Client software. A current version of Chrome, Edge, Firefox, or Safari with JavaScript, IndexedDB, and service worker support. No plugin or installation is required.

2.7 Justification of Platform

The problem addressed by BookBuddy is worth solving for operational, educational, economic, and technical reasons.

Operational justification: A library is only useful if students can find, borrow, read, and use its resources easily. When circulation, digital access, and facility use are separate, every task takes longer than it should and staff spend time on reconciliation rather than service. Paper lab registers are a concrete example. They are routinely incomplete, they cannot be analysed, and they do not stop a student from staying past the permitted hour. Replacing them with verified bookings, automatic release of no-shows, and a waitlist removes a daily source of conflict.

Educational justification: Reading and research are central to learning. In-browser reading with highlights, notes, and cross-device position sync supports how students actually study. Offline access helps students with limited connectivity. Streaks, badges, reviews, and reading lists encourage steady library use, which standalone catalogs do not attempt.

Economic justification: Smaller institutions often cannot afford servers and technical staff for an on-premise system, so they remain on manual processes. A shared cloud deployment lowers the cost per institution, and the use of open-source components and free or low-cost service tiers keeps the total cost within reach. Online fine payment and bulk roster import also reduce staff time.

Security and compliance justification: A shared platform that holds student identities and borrowing history carries real privacy risk. The problem therefore cannot be solved by simply hosting an existing single-tenant system for many colleges. It needs tenant isolation, session theft detection, audit trails, and automated verification, which are core parts of this project.

Technical justification: The chosen stack suits the workload. React and TanStack Query give a responsive interface with efficient caching. MongoDB suits flexible library records and supports compound indexes for tenant-scoped queries. Redis and Socket.io provide real-time updates across instances. HTTP 202 asynchronous processing avoids the timeouts that synchronous imports cause. Atomic updates and transactions protect the state that many users change at once.

Academic justification: The problem exercises a wide range of software engineering skills in one coherent system: requirements analysis, layered architecture, secure authentication,

concurrency control, performance engineering, automated verification, and production deployment.

Conclusion of the analysis: The existing arrangements leave measurable gaps in integration, cost, safety, and student experience. The requirements are clear, the technology is mature, and the work can be planned and verified. A unified, multi-tenant, cloud-native library platform is therefore both necessary and feasible, and BookBuddy is built to meet that need

Chapter 3

System Design

3.1 Module division

BookBuddy is a multi-tenant Integrated Library System (ILS) and digital learning platform. It unifies physical catalog circulation, in-browser EPUB/PDF reading with persistent annotations, computer-lab seat reservation, external catalog harvesting, Razorpay fine settlement, gamified reading habits and asynchronous patron ingestion. To keep a system of this size maintainable, it is divided into modules using the following softwareengineering principles.

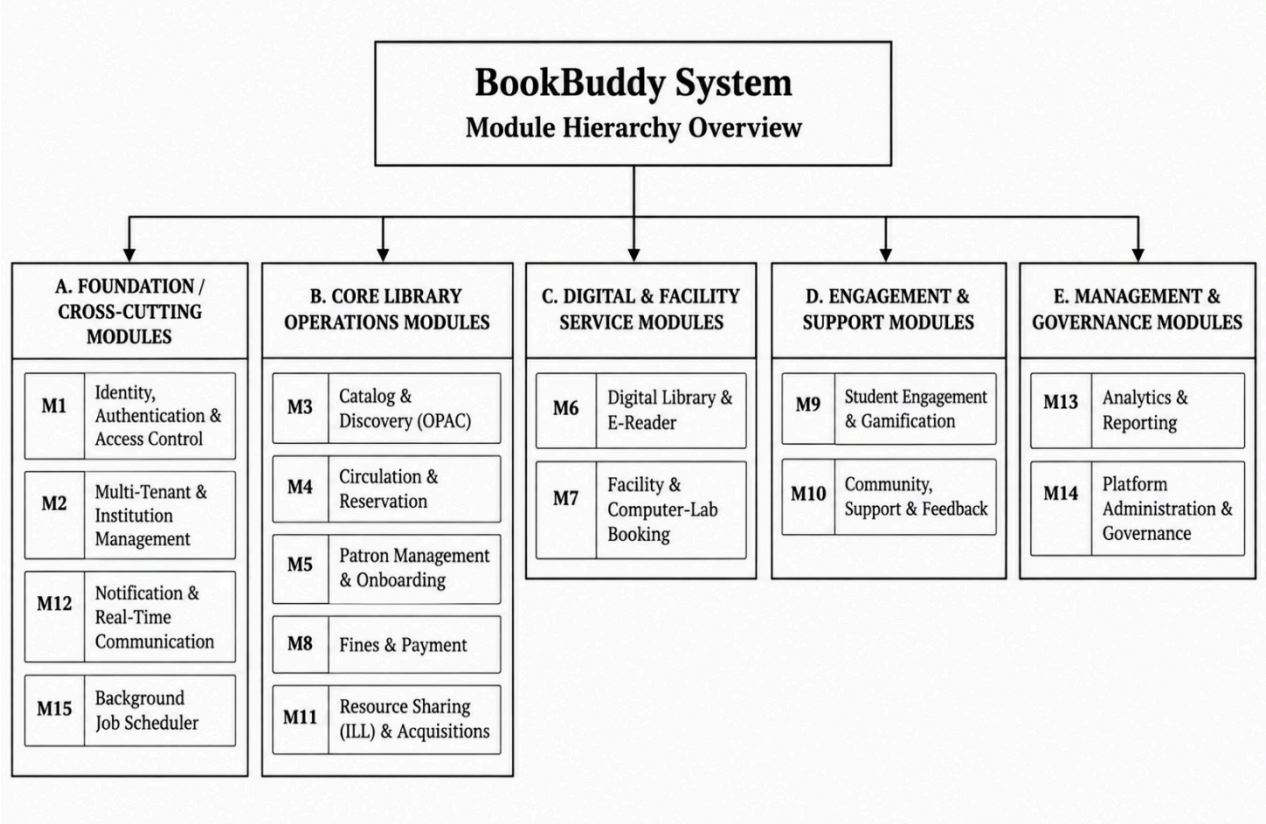
Principle	How it is applied in BookBuddy
Separation of concerns	Each functional area (circulation, digital reader, facilities, payments) has its own controllers, routes, services and Mongoose models.
High cohesion	Every module groups only related responsibilities. For example, all fine and payment logic sits in one module, not scattered across circulation.
Low coupling	Modules communicate through service calls, Socket.io events and shared Zod contracts (@bookbuddy/shared), not by reading each other's internals.
Multi-tenant isolation	A cross-cutting rule: collegeId is injected via req.tenantFilter into every module's queries.
Role-based partitioning	The UI is split into four portals (Public, Student, College Admin, Super Admin), each consuming the same backend modules according to role permissions.

Single source of truth	Roles, status enums and request/response schemas are defined once in the shared workspace and reused by frontend and backend.

Architectural style: a three-tier client–server architecture in a monorepo. The presentation tier is a React 19 SPA, the application tier is an Express 5 REST API with Socket.io, and the data tier is MongoDB, Redis, Cloudinary and Algolia.

2. Module Hierarchy Overview

The BookBuddy System is divided into five major categories, each containing modules responsible for specific system functionalities.



3. Detailed Module Descriptions

M1: Identity, Authentication & Access Control

Purpose: verify who a user is and decide what they may do.

Sub-module	Functionality
Login / Registration	Credential login, student self-registration with college-domain validation, OAuth (Google and GitHub via Passport.js).
Token Management	15-minute JWT access tokens and rotating httpOnly refresh tokens (7 to 30 days), stored in Redis session trees.
Theft Detection	If a refresh token is reused, the whole session family is revoked and all devices are logged out.
CSRF Protection	The _csrf cookie must match the X-CSRF-Token header on every mutating request.
RBAC	Canonical roles (STUDENT , COLLEGE_ADMIN , SUPER_ADMIN) enforced by middleware.
Password Security	Argon2id/bcrypt hashing and forced password change on first login (mustChangePasswordOnNextLogin).
MFA & Impersonation	TOTP MFA for super admins and audited persona impersonation with a warning banner.
Cross-tab Sync	BroadcastChannel keeps token renewal and logout consistent across browser tabs.

Entities: User , RefreshToken , RevokedToken , PendingAdminSetup .

M2: Multi-Tenant & Institution Management

Purpose: give each college a logically isolated workspace on shared infrastructure.

Sub-module	Functionality
Tenant Resolution	Resolves the College from the subdomain (<slug>.bookbuddy.com) or the path /c/:collegeSlug/ .
Tenant Scoping	scopeToTenant.js injects req.tenantFilter = { collegeId } into every query.
Institution Onboarding	Self-service college application and super-admin review queue.
Feature / Service Catalog	Per-college toggles for Gamification, ILL, Feed, Facilities Booking and Razorpay, with transitive dependency resolution and Redis cache purge.
Policy Configuration	Loan period, renewal limit and fine rate per day.
Subscription Tiers	Free, Standard and Enterprise, with active, suspended and archived lifecycle states.

Entities: College , CollegeFeatureConfig , RegistrationRequest .

M3: Catalog & Discovery (OPAC)

Purpose: let users find physical and digital resources.

Sub-module	Functionality
Cataloging Desk	Add or edit books, generate copy barcodes, assign shelf addresses (e.g. Shelf A-12-04).
ISBN Auto-fetch	Pulls metadata and covers from Google Books and Open Library.
OPAC Search	Full-text search with debounced autocomplete and facets (discipline, department, availability, media type), using Algolia and MongoDB.
External Harvesting	Google Books, Open Library and Project Gutenberg clients, with SSRF domain allowlists.
Public Discovery Home	One aggregated endpoint for opening hours, trending books and lab availability.
Guest Bookmarks	localStorage bookmarks for visitors.

Entities: Book , UnifiedBook , OpenLibraryBook .

M4: Circulation & Reservation

Purpose: run the lending lifecycle of physical items.

Sub-module	Functionality
Checkout / Return	Barcode/RFID checkout with atomic copiesAvailable decrement, run inside runInTransaction with an afterCommitFn for side effects.
Renewals	Online renewal up to MAX_RENEWALS (default 2).
Hold Queue	Reserve unavailable titles, show queue position and auto-promote the next patron on return.

Availability Alerts	Notify students when an out-of-stock title is returned.
Inventory Control	Stock audit, condition tracking (Good, Fair, Damaged), lost and damaged reporting.

Entities: Loan , Reservation , AvailabilityAlert , WatchRequest , ItemReport .

M5: Patron Management & Onboarding

Purpose: manage the student roster and library membership.

Sub-module	Functionality
Patron Desk	Searchable roster, loan and fine audit, borrowing suspension.
Bulk CSV Ingestion	HTTP 202 async upload, 500-record chunks via insertMany({ordered:false}) , live progress over Socket.io, error CSV download.
Activation & Join Requests	Single-use activation tokens, roster auto-match, manual approval queue.
Credential Delivery	Email, SMS (Twilio) and printable handouts.
Digital Patron Card	Code128 barcode and QR code.
Gate Scanner API	/patron-card/verify , secured by SCANNER_API_KEY and limited to 60 verifications per minute.

Entities: User , StudentJoinRequest , UploadJob , StudentUploadBatch , UploadAuditLog .

M6: Digital Library & E-Reader

Purpose: provide EPUB/PDF e-resources with a persistent reading experience.

Sub-module	Functionality
E-Resource Repository	Upload, categorise and moderate digital assets, with download analytics.
Streaming Service	HTTP 206 partial-content streaming.
Reader Engine	Epub.js and PDF.js with font scaling (80–160%), Light/Dark/Sepia themes and a table of contents.
Annotations	Colour highlights and sticky notes anchored to CFI or PDF coordinates.
Progress Sync	Reading position and percentage synced across devices.
Offline Manager	IndexedDB (idb) and Service Worker for offline reading.
Student Submissions	Student-contributed notes pass through a moderation queue.

Entities: EResource , EResourceSubmission , Annotation , ReadingProgress , ReadingPosition , DownloadLog .

M7: Facility & Computer-Lab Booking

Purpose: reserve lab workstations and study seats without double-booking.

Sub-module	Functionality
Resource Designer	Bulk seat generation with hardware tags (RAM, GPU, OS, monitors) and maintenance toggles.
Booking Engine	Hourly slots protected by atomicConditionalUpdate against race conditions.
Waitlist Queue	Auto-promotion with a 10-minute confirmation window.
No-Show Control	10-minute grace period, auto-release and a strike counter.
Sub-module	Functionality
Usage Caps	Weekly hour caps aggregated across branches.

Entities: LabSeat , LabBooking , FacilityResource , FacilityResourceGroup , FacilityBooking , FacilityBookingQueue , NoShowStrike , FacilityUsageWeekly .

M8: Fines & Payment

Purpose: calculate overdue penalties and collect them.

Sub-module	Functionality
Fine Accrual	Daily overdue computation (FINE_RATE_PER_DAY , capped at FINE_MAX_AMOUNT).
Online Payment	Razorpay order creation and HMAC-SHA256 signature verification.
Webhook Processor	Idempotent processing on raw request buffers.
Counter Settlement	Cash receipts and waivers with a mandatory justification note, written to AuditLog .
Reconciliation	Nightly comparison of local Payment records with the Razorpay API.

Entities: Fine , Payment , AuditLog .

M9: Student Engagement & Gamification Purpose: encourage regular library use.

Sub-module	Functionality
Streak Engine	Daily check-in, streak freeze tokens and automatic expiry sweep.
Badges & Stickers	Milestone achievements.
Leaderboard	Campus ranking and weekly snapshots.
Reading Lists & Shelves	Custom shelves with drag-and-drop ordering (@dnd-kit).
Reviews & Ratings	1–5 stars, helpfulness votes and moderation.
Sub-module	Functionality
Recommendations	Nightly personalised suggestions by major, borrowing history and popularity.

Entities: Streak , CheckInLog , StreakReward , Badge , Sticker , UserBadge , UserSticker , Review , Shelf , UserRecommendation , LeaderboardSnapshot .

M10: Community, Support & Feedback

Purpose: handle communication between patrons and staff.

Sub-module	Functionality
Community Feed	Campus bulletin and discussion posts.
Helpdesk	Complaints and tickets with staff notes and status updates.
Purchase Suggestions	Patron acquisition requests.
Announcements	Live ticker over Socket.io.
Escalation	College tickets escalated to the super-admin support queue.

Entities: FeedPost , Complaint , BookSuggestion , ItemReport .

M11: Resource Sharing (ILL) & Acquisitions

Purpose: extend holdings through partner colleges and procurement.

Sub-module	Functionality
Cross-College Catalog	Search partner institutions in the consortium.
ILL Workflow	Request, approve, ship (with tracking number) and confirm return.
Purchase Orders	Create orders, compare vendor quotes and track budgets.
Receiving & Serials	ISBN receiving batches and subscription renewals, gated by canManageAcquisitions .

Entities: ILLRequest , ShareRequest , AcquisitionOrder , BookRequest .

M12: Notification & Real-Time Communication Purpose: deliver timely information over multiple channels.

Sub-module	Functionality
Socket.io Gateway	Rooms college: , user: , desk:circulation: , desk:inventory: , desk:patrons: and bulkupload: , with the Redis adapter for clustering.
Runtime Kill Switch	realtimeFlagService.js flags throttle broadcasting without a restart.
Query Cache Patcher	Applies socket events directly to TanStack Query caches.
Multi-channel Delivery	In-app, email (Nodemailer) and SMS (Twilio) according to user preferences.

Entities: Notification , DeviceToken .

M13: Analytics & Reporting

Purpose: turn operational data into decisions.

Sub-module	Functionality
College Analytics	Circulation velocity, top borrowed books, department distribution, peak lab hours.
Snapshot Aggregation	Nightly precomputed metrics in DashboardStatsSnapshot .
Report Export	CSV export (/reports/:type), neutralised against formula injection.
Platform Oversight	Cross-tenant comparisons and storage-growth forecasting (moving average).

Entities: DashboardStatsSnapshot , PlatformMetricSnapshot .

M14: Platform Administration & Governance

Purpose: give the super admin control over the whole platform.

Sub-module	Functionality
System Overview & Health	KPIs, cluster telemetry, /health and /version .
College Admin Manager	Provision tenants and the first admin atomically.
Onboarding Review Queue	Atomic approve/reject of college applications.
Global Content Moderation	Approve or reject e-resources, reviews and posts.
User Directory	Global search, role changes, deactivation and session revocation.
Audit Logs	Immutable, filterable and exportable.
System Settings & DR	Rate limits, SMTP, maintenance mode, and database restore (requires CONFIRM_RESTORE and MFA step-up).
Global Support Queue	Platform-level ticket resolution.

Entities: AuditLog , CronRunLog , College , User .

M15: Background Job Scheduler

Purpose: run time-based automation reliably (node-cron , with execution locks and run logging).

Job group	Jobs
Circulation	Overdue fine accrual (daily), hold expiry sweep (every 15 min), due-date reminders (daily).
Gamification	Streak expiry sweep, streak cutoff reminders, weekly leaderboard snapshot, nightly recommendations.
Facilities	No-show auto-release (every 2 min), queue promotion expiry (every 1 min).
Finance & Maintenance	Payment reconciliation, stale registration cleanup, metrics aggregation.

Entities: CronRunLog , plus the models each job touches.

4. Module-to-Actor Access Matrix

Module	Public Visitor	Student	College Admin	Super Admin
M1 Auth & Access	Yes (login/register)	Yes	Yes	Yes (MFA, impersonation)
M2 Tenant & Institution	Yes (apply)	No	Yes (features/policy)	Yes (full)
M3 Catalog & Discovery	Yes	Yes	Yes (manage)	Yes
M4 Circulation & Reservation	No	Yes (holds, renew)	Yes (checkout/return)	Yes
M5 Patron Management	No	Yes (own card)	Yes	Yes
M6 Digital Library	Yes (preview)	Yes	Yes (upload)	Yes (moderate)

M7 Facility Booking	No	Yes	Yes (configure)	Yes
M8 Fines & Payment	No	Yes (pay)	Yes (waive/cash)	Yes
M9 Engagement	No	Yes	No	No
M10 Community & Support	No	Yes	Yes (resolve)	Yes (escalations)
M11 ILL & Acquisitions	No	Yes (ILL request)	Yes	Yes
M12 Notifications	Yes (ticker)	Yes	Yes	Yes
M13 Analytics	No	No	Yes	Yes (global)
M14 Platform Admin	No	No	No	Yes
M15 Scheduler	No	No	No	Yes (monitor)

5. Inter-Module Dependencies

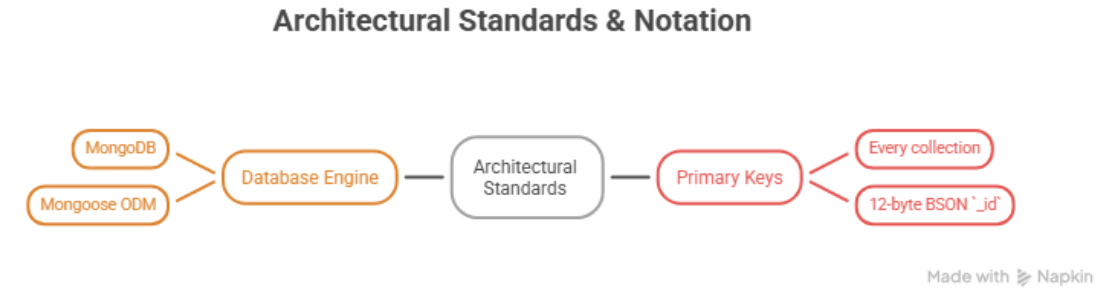
Calling module	Depends on	Nature of dependency
All modules	M1, M2	Every request passes authentication, CSRF, tenant scoping and the feature-flag guard.
M4 Circulation	M3, M5, M8, M12	Needs book copies and patron status; creates fines on return; sends hold-ready notices.
M8 Fines	M4, M12	Fines come from overdue loans; payment status is pushed to the user.
M7 Facilities	M5, M12	Student identity and strikes; waitlist promotion alerts.
M6 Digital Library	M3, M10	Shares catalog metadata; moderation of submissions.

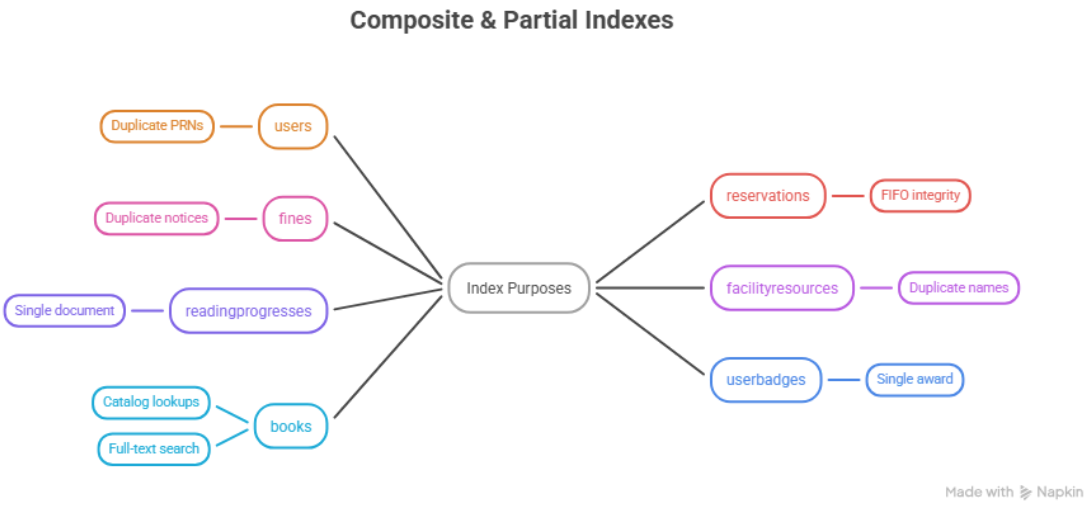
M9 Engagement	M4, M6, M12	Streaks and recommendations use loans and reading activity.
M13 Analytics	M4, M5, M7, M8	Reads data from the operational modules.
M15 Scheduler	M4, M7, M8, M9, M12	Triggers their time-based logic.

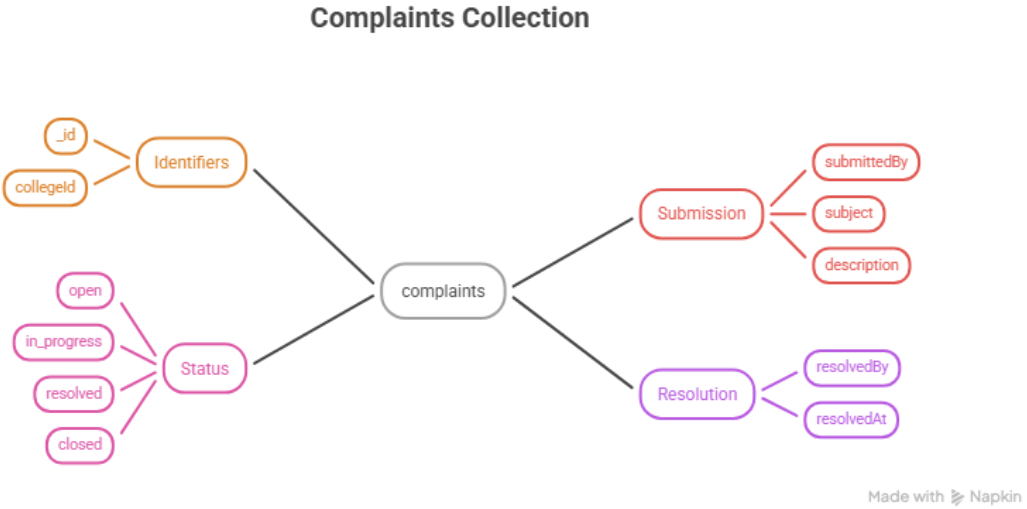
6. Layered View of the Modules

Layer	Components	Responsibility
Presentation	React pages and components, Zustand stores, TanStack Query, SocketContext	UI rendering, client state, cache patching.
API / Middleware	Auth, CSRF, scopeToTenant , Feature Gate, Rate Limiter, Zod validate.js	Cross-cutting security and validation.
Business Logic	Controllers and services (cronService , realtimeFlagService)	Module rules and transactions.
Data Access	73 Mongoose models, atomicUpdateHelper , transactionHelper	Persistence, atomic updates and tenant-prefixed compound indexes.
Infrastructure	MongoDB Atlas, Redis, Cloudinary, Algolia, Razorpay, Twilio, SMTP	External and persistent services.

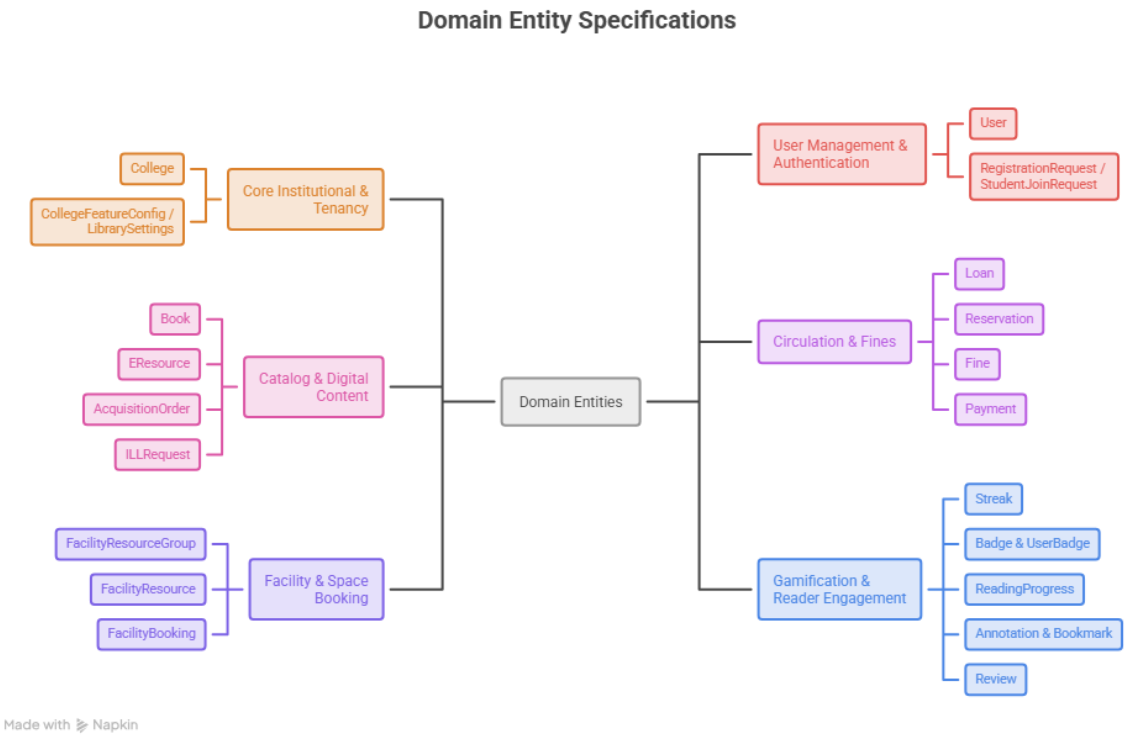
3.2 Data dictionary

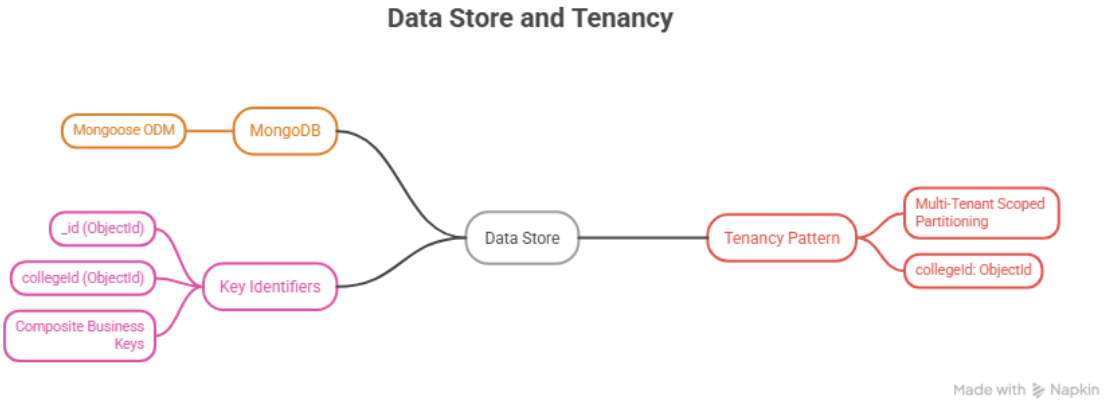


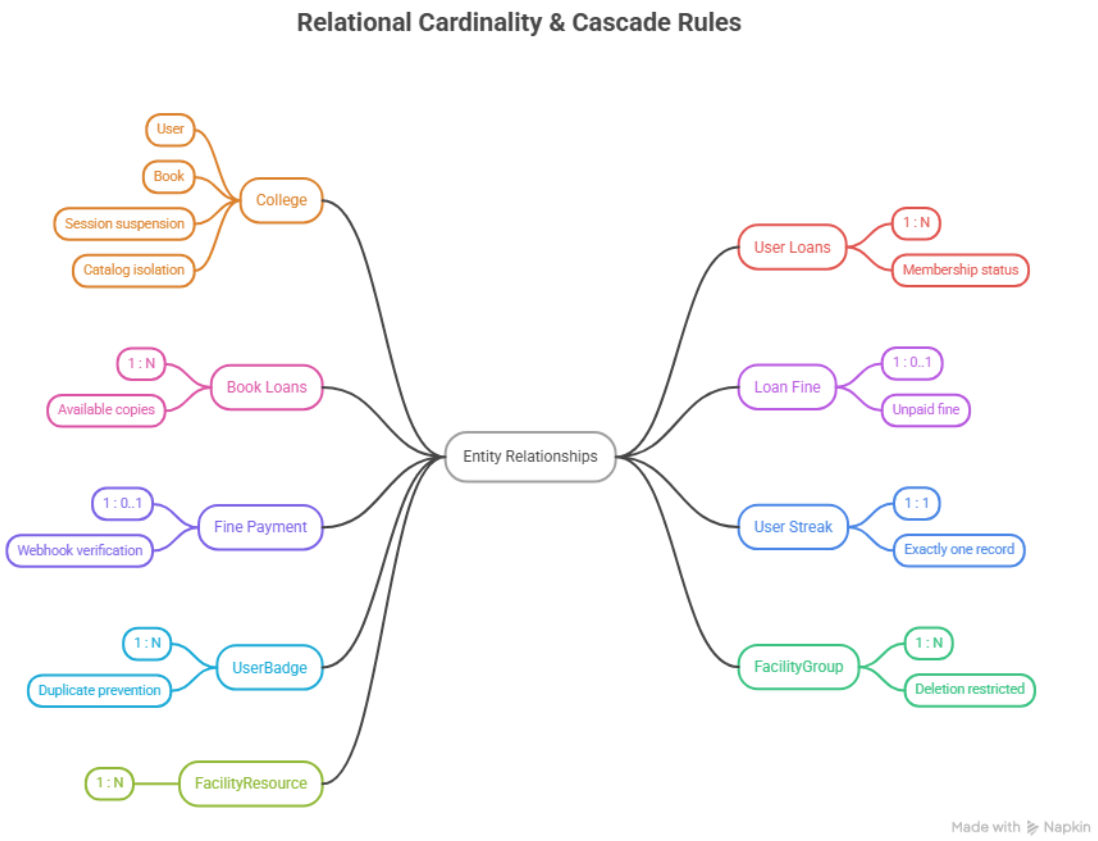




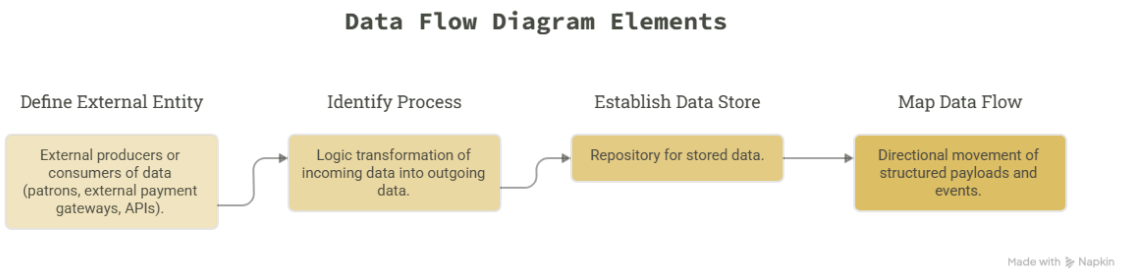
3.3 E-R Diagrams



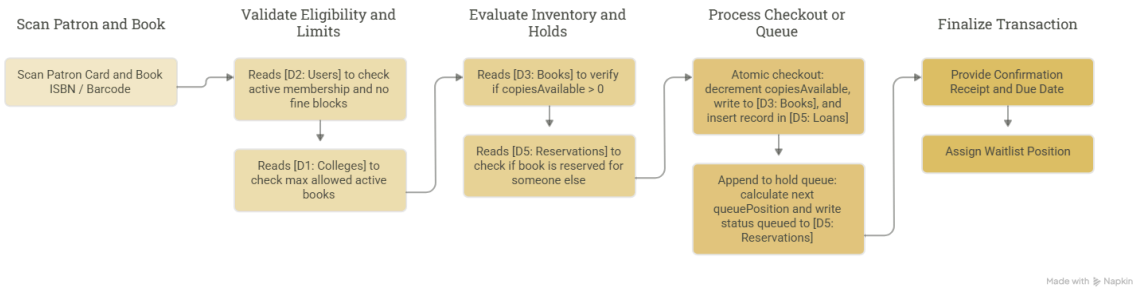




3.4 Data Flow Diagrams.

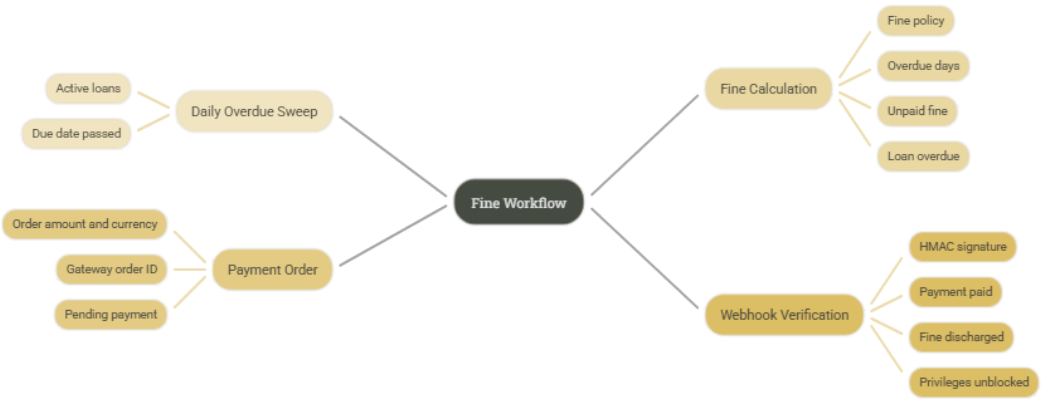


Library Book Checkout and Reservation Process



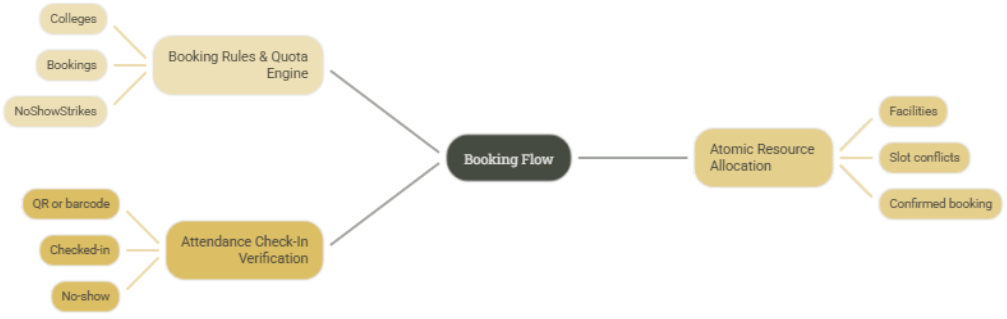
Made with Napkin

Overdue Fines and Payments



Made with Napkin

Booking and Check-In Flow



Made with Napkin

Chapter 4 Implementation and Testing

4.1 Code (Core Segments) :

index.js:

```
const connectDB = require('../backend/src/db');

const app = require('../backend/src/app');



module.exports = async (req, res) => {

  try {

    await connectDB();

  } catch (err) {

    // eslint-disable-next-line no-console

    console.warn('[Vercel Serverless DB Connection Warning]:',
err.message);

  }



  // Ensure request URL has valid /api prefix for Express router
matching

  if (req.url && !req.url.startsWith('/api')) {

    req.url = `/api${req.url.startsWith('/') ? '' : '/'}${req.url}`;

  }

  if (!req.originalUrl) {

    req.originalUrl = req.url;

  }



  return app(req, res);

};
```

db.js:

```
// Database connection manager with retry logic and event logging.

const mongoose = require('mongoose');

const config = require('./index');

const env = require('./env');

const logger = require('../utils/logger');


// Set strictQuery rule

mongoose.set('strictQuery', true);


// Monitor connection events

mongoose.connection.on('connected', () => {

  const host = mongoose.connection.host || 'unknown-host';

  const name = mongoose.connection.name || 'unknown-db';

  logger.info(`Mongoose connection status: Connected to host=${host},
database=${name}`);

});


mongoose.connection.on('reconnected', () => {

  logger.info('Mongoose connection status: Reconnected.');

});


mongoose.connection.on('error', (err) => {

  logger.error(`Mongoose connection status: Error: ${err.message}`);

});
```

```
mongoose.connection.on('disconnected', () => {
  logger.warn('Mongoose connection status: Disconnected.');
});

const connectDB = async () => {
  const mongoUri = process.env.MONGO_URI;

  if (!mongoUri) {
    const errorMsg =
      'MONGO_URI is not set — check environment variables (Production / Preview / Development) for this environment.';
    logger.error(`[Database Boot Failure] ${errorMsg}`);
    throw new Error(errorMsg);
  }

  if (!global._mongooseConn) {
    global._mongooseConn = { conn: null, promise: null };
  }
  const cached = global._mongooseConn;

  if (cached.conn && mongoose.connection.readyState === 1) {
    return cached.conn;
  }

  if (mongoose.connection.readyState === 1) {
    cached.conn = mongoose;
    return cached.conn;
```

```
  }


  if (cached.promise) {
    try {
      cached.conn = await cached.promise;
      return cached.conn;
    } catch {
      cached.promise = null;
    }
  }


  const defaultPoolSize = env.VERCEL ? 5 : 10;
  const options = {
    maxPoolSize: env.MONGO_MAX_POOL_SIZE || defaultPoolSize,
    serverSelectionTimeoutMS: 15000,
  };


  const isProd = config.nodeEnv === 'production' || !!env.RENDER;
  let primaryUri = mongoUri || config.mongoUri;
  if (
    process.env.NODE_ENV === 'test' &&
    global.__ORIGINAL_ATLAS_TEST_URI__ &&
    (!primaryUri || primaryUri.includes('localhost') ||
primaryUri.includes('127.0.0.1'))
  ) {
    primaryUri = global.__ORIGINAL_ATLAS_TEST_URI__;
  }
```

```
  // Fall back to local MongoDB ONLY if no explicit MONGO_URI was
specified in environment variables

  const allowLocalFallback = !mongoUri && !isProd;

  const fallbackUris = allowLocalFallback ?
['mongodb://localhost:27017/bookbuddy'] : [];



  const targetUris = Array.from(new Set([primaryUri,
...fallbackUris])).filter(Boolean);



  let lastError = null;

  cached.promise = (async () => {

    for (const uri of targetUris) {

      try {

        const sanitizedUri = uri.includes('@')

          ? uri.replace(/\/\/[^:]+:[^@]+@/, '//***:***@')

          : uri;

        logger.info(`Attempting MongoDB connection to
${sanitizedUri}...`);

        const mongooseInstance = await mongoose.connect(uri, options);

        const connHost = mongoose.connection.host || 'unknown';

        const connDb = mongoose.connection.name || 'unknown';

        logger.info(`MongoDB connected successfully to host
[${connHost}], database [${connDb}].`);

        return mongooseInstance;

      } catch (err) {

        lastError = err;

        const displayHost = uri.includes('@') ? uri.split('@')[1] :
uri;
```

```
        logger.error(`MongoDB connection to ${displayHost} failed:
${err.message}`);

      }

    }

    throw lastError || new Error('All MongoDB connection attempts
failed');

  })();


  try {

    cached.conn = await cached.promise;

    return cached.conn;

  } catch (err) {

    cached.promise = null;

    logger.error(

      `❌ FATAL MONGODB CONNECTION ERROR (${config.nodeEnv}): ${err ?
err.message : 'Unknown error'}`

    );

    throw err;

  }

};


module.exports = connectDB;
```

env.js:

```
const dotenv = require('dotenv');

const path = require('path');

const { z } = require('zod');
```

```
// Load environment variables from backend/.env
dotenv.config({ path: path.join(__dirname, '../../.env') });

// Load bundled secrets (SECRETS_JSON or /etc/bookbuddy/secrets.json)
if present
const { loadSecrets } = require('./secretsLoader');
loadSecrets();

// Support MONGODB_URI alias (Vercel / Mongo Atlas standard)
if (!process.env.MONGO_URI && process.env.MONGODB_URI) {
  process.env.MONGO_URI = process.env.MONGODB_URI;
}

const isTest = process.env.NODE_ENV === 'test';

const requiredString = (name, customMsg, minLen = 1) =>
  z.preprocess(
    (val) => (val === '' || val === null ? undefined : val),
    z
      .string({
        required_error: customMsg || `${name} is required`,
        invalid_type_error: customMsg || `${name} is required`,
      })
      .min(minLen, customMsg || `${name} must be at least ${minLen}
characters long`)
  );
```

```
const KNOWN_DEFAULT_SECRETS = [
  'secret',
  'jwtsecret',
  'yoursecretkey123',
  'yourrefreshsecretkey456',
  'supersecretrefresh123',
  'supersecret123',
  'supersecretsession123',
  'testjwtsecretkey999',
  'password',
  'password123',
  'admin',
  'admin123',
  'changeme',
  '123456',
  '12345678',
  '1234567890',
  'devsuperadminpass@2026!',
  'bookbuddy_scanner_secret_2026',
  'placeholder_razorpay_secret_key',
  'rzp_test_placeholder_id',
  'default_jwt_secret',
  'default_secret',
  // README & Documentation Sample Placeholders
  'super_secure_jwt_access_secret_passphrase_min_32_chars',
  'super_secure_jwt_refresh_secret_passphrase_min_32_chars',
  'super_secure_cookie_signing_secret_min_32_chars',
```

```
  'replace_with_strong_32_char_jwt_secret_in_production',
  'replace_with_strong_32_char_jwt_refresh_secret_in_production',
  'replace_with_strong_32_char_cookie_secret_in_production',
  'replace_with_strong_16_char_scanner_key',
  'your_google_oauth_client_id',
  'your_google_oauth_client_secret',
  'your_github_oauth_client_id',
  'your_github_oauth_client_secret',
  'rzp_test_your_razorpay_key_id',
  'your_razorpay_secret_key',
  'your_razorpay_webhook_secret',
  'your_cloudinary_name',
  'your_cloudinary_api_key',
  'your_cloudinary_api_secret',
  'vercel_blob_token_if_deployed_on_vercel',
  'app_specific_smtp_password',
  'ac_your_twilio_account_sid',
  'your_twilio_auth_token',
  'your_algolia_app_id',
  'your_algolia_admin_key',
  'your_google_books_api_key',
];


const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'production',
'test']).default('development'),
  PORT: z.coerce.number().int().positive().default(5000),
```

```
  HOST: z.string().default('0.0.0.0'),


  MONGO_URI: isTest
    ? z.string().default('mongodb://127.0.0.1:27017/bookbuddy_test')
    : requiredString('MONGO_URI', 'MONGO_URI is required').refine(
        (val) => val.startsWith('mongodb://') ||
val.startsWith('mongodb+srv://'),
        {
          message:
            'MONGO_URI must be a valid connection string starting with
mongodb:// or mongodb+srv://',
        }
      ),


  JWT_SECRET: isTest
    ? z.string().min(1).default('testjwtsecretkey999')
    : requiredString(
        'JWT_SECRET',
        'JWT_SECRET is required and must be at least 32 characters (256
bits) in production',
        process.env.NODE_ENV === 'production' ? 32 : 8
      ).refine((val) =>
!KNOWN_DEFAULT_SECRETS.includes(val.toLowerCase()), {
        message: 'JWT_SECRET must not use a weak or default value',
      }),


  JWT_REFRESH_SECRET: isTest
    ? z.string().min(1).default('supersecretrefresh123')
```

```
    : requiredString(
        'JWT_REFRESH_SECRET',
        'JWT_REFRESH_SECRET is required and must be at least 32 characters (256 bits) in production',
        process.env.NODE_ENV === 'production' ? 32 : 8
      ).refine((val) => !KNOWN_DEFAULT_SECRETS.includes(val.toLowerCase()), {
        message: 'JWT_REFRESH_SECRET must not use a weak or default value',
      }),
  JWT_ACCESS_EXPIRY: z.string().default('15m'),
  JWT_REFRESH_EXPIRY: z.string().default('7d'),
  JWT_ISSUER: z.string().default('bookbuddy-api'),
  JWT_AUDIENCE: z.string().default('bookbuddy-client'),

  CLIENT_ORIGIN: z.string().default('*'),
  CLIENT_URL: z.string().optional(),

  REDIS_URL: isTest
    ? z.string().default('redis://127.0.0.1:6379')
    : requiredString(
        'REDIS_URL',
        'REDIS_URL is required — this app relies on Redis for multi-instance Socket.io'
      ),

  // Cloudinary
  CLOUDINARY_CLOUD_NAME: z.string().optional(),
```

```
  CLOUDINARY_API_KEY: z.string().optional(),
  CLOUDINARY_API_SECRET: z.string().optional(),


  // Sentry
  SENTRY_DSN: z.string().optional(),


  // Mailer
  SMTP_HOST: z.string().optional(),
  SMTP_PORT: z.preprocess(
    (val) => (val === '' || val === null || val === undefined ?
undefined : Number(val)),
    z
      .number()
      .refine((val) => val !== 25, {
        message: 'SMTP port 25 is insecure; use 587 or 465',
      })
      .optional()
  ),
  SMTP_USER: z.string().optional(),
  SMTP_PASS: z.string().optional(),
  SMTP_FROM: z.string().optional(),


  // Algolia
  ALGOLIA_APP_ID: z.string().optional(),
  ALGOLIA_ADMIN_KEY: z.string().optional(),
  ALGOLIA_INDEX_NAME: z.string().default('bookbuddy_books'),
```

```
  // Google OAuth & Books
  GOOGLE_CLIENT_ID: z.string().optional(),
  GOOGLE_CLIENT_SECRET: z.string().optional(),
  GOOGLE_CALLBACK_URL: z.string().optional(),
  GOOGLE_BOOKS_API_KEY: z.string().optional(),

  // GitHub OAuth
  GITHUB_CLIENT_ID: z.string().optional(),
  GITHUB_CLIENT_SECRET: z.string().optional(),
  GITHUB_CALLBACK_URL: z.string().optional(),

  // Razorpay
  RAZORPAY_KEY_ID: isTest ?
z.string().default('rzp_test_placeholder_id') : z.string().optional(),
  RAZORPAY_KEY_SECRET: isTest
    ? z.string().default('placeholder_razorpay_secret_key')
    : z.string().optional(),
  RAZORPAY_WEBHOOK_SECRET: z.string().optional(),

  // Business logic settings
  LOAN_PERIOD_DAYS: z.coerce.number().default(14),
  MAX_RENEWALS: z.coerce.number().default(2),
  UNPAID_FINE_LIMIT: z.coerce.number().default(100),
  FINE_RATE_PER_DAY: z.coerce.number().default(5),
  FINE_MAX_AMOUNT: z.coerce.number().default(100),
  HOLD_PICKUP_WINDOW_HOURS: z.coerce.number().default(48),
  DUE_REMINDER_DAYS_BEFORE: z.coerce.number().default(2),
```

```
  STREAK_REMINDER_HOURS_BEFORE: z.coerce.number().default(3),
  LAB_START_HOUR: z.coerce.number().default(8),
  LAB_END_HOUR: z.coerce.number().default(20),

  // Rate Limits
  RATE_LIMIT_GLOBAL_MAX: z.coerce.number().default(isTest ? 10000 :
500),
  RATE_LIMIT_GLOBAL_WINDOW_MS: z.coerce.number().default(60000),
  RATE_LIMIT_AUTH_MAX: z.coerce.number().default(isTest ? 10000 : 15),
  RATE_LIMIT_AUTH_IP_MAX: z.coerce.number().default(isTest ? 10000 :
60),
  RATE_LIMIT_AUTH_EMAIL_MAX: z.coerce.number().default(isTest ? 10000 :
15),
  RATE_LIMIT_AUTH_WINDOW_MS: z.coerce.number().default(900000),
  RATE_LIMIT_USER_MAX: z.coerce.number().default(isTest ? 10000 : 300),
  RATE_LIMIT_USER_WINDOW_MS: z.coerce.number().default(60000),
  RATE_LIMIT_EXPENSIVE_MAX: z.coerce.number().default(isTest ? 10000 :
30),
  RATE_LIMIT_EXPENSIVE_WINDOW_MS: z.coerce.number().default(60000),
  RATE_LIMIT_SEARCH_MAX: z.coerce.number().default(isTest ? 10000 :
60),
  RATE_LIMIT_SEARCH_WINDOW_MS: z.coerce.number().default(60000),
  RATE_LIMIT_PATRON_CARD_VERIFY_MAX: z.coerce.number().default(isTest ?
10000 : 60),
  RATE_LIMIT_PATRON_CARD_VERIFY_WINDOW_MS:
z.coerce.number().default(60000),
  LOGIN_LOCKOUT_MAX_ATTEMPTS: z.coerce.number().default(5),
  LOGIN_LOCKOUT_DURATION_S: z.coerce.number().default(900),
  LOGIN_LOCKOUT_WINDOW_S: z.coerce.number().default(900),
```

```
  TRUST_PROXY: z.union([z.boolean(), z.number(),
z.string()]).default(false),

  TRUST_PROXY_HOPS: z.coerce.number().int().min(0).default(1),

  SOCKET_IO_REDIS_ADAPTER_ENABLED: z

    .preprocess((val) => {

      if (val === 'false' || val === false || val === '0') return
false;

      if (val === 'true' || val === true || val === '1') return true;

      return val;

    }, z.boolean())

    .default(true),

  ENABLE_CRON: z

    .preprocess((val) => {

      if (val === 'false' || val === false || val === '0') return
false;

      if (val === 'true' || val === true || val === '1') return true;

      return val;

    }, z.boolean())

    .default(true),

  TIMEZONE_AWARE_CRON_ENABLED: z

    .preprocess((val) => {

      if (val === 'false' || val === false || val === '0') return
false;

      if (val === 'true' || val === true || val === '1') return true;

      return val;

    }, z.boolean())

    .default(true),
```

```
  // Scanner Gate Verification Security
  SCANNER_API_KEY: z.string().default('bookbuddy_scanner_secret_2026'),
  COOKIE_SECRET: z.string().optional(),

  // Deployment / Platform flags
  VERCEL: z.string().optional(),
  RENDER: z.string().optional(),
  MONGO_MAX_POOL_SIZE: z.coerce.number().optional(),
  ERROR_WEBHOOK_URL: z.string().optional(),
  SLACK_WEBHOOK_URL: z.string().optional(),
  SIEM_WEBHOOK_URL: z.string().optional(),
  BLOB_READ_WRITE_TOKEN: z.string().optional(),
  STORAGE_PROVIDER: z.enum(['local', 'gcs', 's3',
'vercel-blob']).optional(),
  STORAGE_SIGNING_SECRET: z.string().optional(),
  STORAGE_LOCAL_DIR: z.string().optional(),
  MAX_EBOOK_UPLOAD_SIZE_BYTES: z.coerce.number().default(50 * 1024 *
1024),
  MAX_ROSTER_UPLOAD_SIZE_BYTES: z.coerce.number().default(20 * 1024 *
1024),
  ENABLE_MEMORY_LOGGING: z.string().optional(),
  APP_VERSION: z.string().optional(),
  RENDER_GIT_COMMIT: z.string().optional(),
  VERCEL_GIT_COMMIT_SHA: z.string().optional(),
  COMMIT_SHA: z.string().optional(),
  GITHUB_SHA: z.string().optional(),
  BUILD_ID: z.string().optional(),
  GUTENDEX_BASE_URL: z.string().optional(),
```

```
  GUTENDEX_CACHE_TTL_SECONDS: z.coerce.number().optional(),
  GUTENDEX_TIMEOUT_MS: z.coerce.number().optional(),
  OPEN_LIBRARY_USER_AGENT: z.string().optional(),
  ROTATION_GRACE_PERIOD_MS: z.coerce.number().optional(),
  EMAIL_DRIVER_THROW: z.string().optional(),
  PUSH_DRIVER_THROW: z.string().optional(),
  TEST_CSRF: z.string().optional(),
  TEST_AUDIT_IMMUTABLE: z.string().optional(),
  INSTANCE_ID: z.string().optional(),
  PM2_ID: z.string().optional(),
  HEALTH_CHECK_TOKEN: z.string().optional(),
  TRUSTED_PROXIES: z.string().optional(),
  ADDITIONAL_CORS_ORIGINS: z.string().optional(),

  // Twilio / SMS Integration & Indian DLT
  TWILIO_SID: z.string().optional(),
  TWILIO_AUTH_TOKEN: z.string().optional(),
  TWILIO_FROM_NUMBER: z.string().optional(),
  SMS_PROVIDER: z.string().optional(),
  DLT_ENTITY_ID: z.string().optional(),
  DLT_DEFAULT_SENDER_HEADER: z.string().optional(),
  DLT_TEMPLATE_ONBOARDING: z.string().optional(),
  DLT_TEMPLATE_OVERDUE: z.string().optional(),
  DLT_TEMPLATE_RESERVATION: z.string().optional(),
  DLT_TEMPLATE_OTP: z.string().optional(),

  // OAuth Unification
```

```
  OAUTH_STATE_SECRET: z.string().optional(),
  OAUTH_STATE_SECRET_PREVIOUS: z.string().optional(),
  AUTH_DOMAIN: z.string().optional(),

  // Cloudflare Origin Isolation & Firewall Enforcement (Step 37)
  CLOUDFLARE_ENFORCE: z.string().optional(),
  CLOUDFLARE_ORIGIN_SECRET: z.string().optional(),

  // Zero-Downtime Secret Rotation Readiness (Step 38)
  JWT_SECRET_PREVIOUS: z.string().optional(),
  JWT_REFRESH_SECRET_PREVIOUS: z.string().optional(),
  STORAGE_SIGNING_SECRET_PREVIOUS: z.string().optional(),
});

function checkDefaultCredentials(envData) {
  if (envData.NODE_ENV !== 'production') {
    return;
  }

  const errors = [];

  const isKnownDefault = (val) => {
    if (!val || typeof val !== 'string') return false;
    const lower = val.toLowerCase().trim();
    return KNOWN_DEFAULT_SECRETS.includes(lower);
  };
```

```
  if (isKnownDefault(envData.JWT_SECRET)) {
    errors.push('JWT_SECRET cannot be a default or known weak secret in production');
  }

  if (isKnownDefault(envData.JWT_REFRESH_SECRET)) {
    errors.push('JWT_REFRESH_SECRET cannot be a default or known weak secret in production');
  }

  if (
    envData.JWT_SECRET &&
    envData.JWT_REFRESH_SECRET &&
    envData.JWT_SECRET === envData.JWT_REFRESH_SECRET
  ) {
    errors.push('JWT_REFRESH_SECRET must be different from JWT_SECRET in production');
  }

  if (isKnownDefault(envData.COOKIE_SECRET)) {
    errors.push('COOKIE_SECRET cannot be a default or known weak secret in production');
  }

  if (envData.COOKIE_SECRET && envData.COOKIE_SECRET.length < 32) {
    errors.push('COOKIE_SECRET must be at least 32 characters (256 bits) in production');
  }
```

```
  if (isKnownDefault(envData.SCANNER_API_KEY)) {
    errors.push('SCANNER_API_KEY must not use the default fallback key in production');
  } else if (envData.SCANNER_API_KEY && envData.SCANNER_API_KEY.length < 16) {
    errors.push('SCANNER_API_KEY must be at least 16 characters in production');
  }

  if (envData.MONGO_URI) {
    const defaultMongoCreds = [
      'admin:admin@',
      'root:root@',
      'root:password@',
      'user:password@',
      'mongo:mongo@',
      'mongodb://127.0.0.1:27017/bookbuddy_test',
    ];
    if (defaultMongoCreds.some((cred) => envData.MONGO_URI.toLowerCase().includes(cred))) {
      errors.push('MONGO_URI contains default or unauthenticated test database credentials');
    }
  }

  const superAdminPass = process.env.SEED_SUPER_ADMIN_PASSWORD || process.env.SUPERADMIN_PASSWORD;
  if (superAdminPass && isKnownDefault(superAdminPass)) {
```

```
    errors.push('SEED_SUPER_ADMIN_PASSWORD cannot be a known default
password in production');

  }



  if (envData.RAZORPAY_KEY_SECRET &&
isKnownDefault(envData.RAZORPAY_KEY_SECRET)) {

    errors.push('RAZORPAY_KEY_SECRET cannot use default or placeholder
credentials in production');

  }



  if (errors.length > 0) {

    throw new Error(

      `❌ Production Security Violation: Default credentials
detected:\n  - ${errors.join('\n  - ')}`

    );

  }

}



function validateEnv() {

  const result = envSchema.safeParse(process.env);

  if (!result.success) {

    const fieldErrors = result.error.flatten().fieldErrors;

    const errorDetails = Object.entries(fieldErrors)

      .map(([field, errors]) => `${field}: ${errors.join(', ')}`)

      .join('; ');



    // If running unit tests expecting error throwing or process.exit

    if (process.env.NODE_ENV === 'test' || process.env.JEST_WORKER_ID
!== undefined) {
```

```
      throw new Error(`Environment validation failed:
${errorDetails}`);
    }

    // eslint-disable-next-line no-console
    console.error('\n❌ FATAL: Environment variable validation
failed:');
    for (const [field, errors] of Object.entries(fieldErrors)) {
      // eslint-disable-next-line no-console
      console.error(`  - ${field}: ${errors.join(', ')}`);
    }
    // eslint-disable-next-line no-console
    console.error(
      '\nPlease fix the environment configuration in backend/.env
before starting the server.\n'
    );
    process.exit(1);
  }

  const data = result.data;

  // Enforce default credential prohibition in production
  checkDefaultCredentials(data);

  // Additional production validation check
  if (data.NODE_ENV === 'production' && !data.VERCEL && !data.RENDER) {
    const prodRequired = ['GOOGLE_BOOKS_API_KEY', 'REDIS_URL'];
```

```
    const missingProdEnv = prodRequired.filter((key) =>
!process.env[key]);

    if (missingProdEnv.length > 0) {

      throw new Error(

        `❌ Fatal Startup Error: Missing required production
environment variables: [${missingProdEnv.join(', ')}].`

      );

    }

  }


  return data;

}


const env = validateEnv();

env.checkDefaultCredentials = checkDefaultCredentials;


module.exports = env;
```

index.js:

```
// Centralized environment and configuration manager.

const env = require('./env');


const config = {

  port: env.PORT,

  host: env.HOST,
```

```
mongoUri: env.MONGO_URI,
jwt: {
  secret: env.JWT_SECRET,
  refreshSecret: env.JWT_REFRESH_SECRET,
  accessExpiry: env.JWT_ACCESS_EXPIRY,
  refreshExpiry: env.JWT_REFRESH_EXPIRY,
  issuer: env.JWT_ISSUER || 'bookbuddy-api',
  audience: env.JWT_AUDIENCE || 'bookbuddy-client',
},
clientOrigin: env.CLIENT_ORIGIN,
nodeEnv: env.NODE_ENV,
loanPeriodDays: env.LOAN_PERIOD_DAYS,
maxRenewals: env.MAX_RENEWALS,
unpaidFineLimit: env.UNPAID_FINE_LIMIT,
fineRatePerDay: env.FINE_RATE_PER_DAY,
fineMaxAmount: env.FINE_MAX_AMOUNT,
holdPickupWindowHours: env.HOLD_PICKUP_WINDOW_HOURS,
dueReminderDaysBefore: env.DUE_REMINDER_DAYS_BEFORE,
streakReminderHoursBefore: env.STREAK_REMINDER_HOURS_BEFORE,
labOperatingHours: {
  startHour: env.LAB_START_HOUR,
  endHour: env.LAB_END_HOUR,
},
redisUrl: env.REDIS_URL,
rateLimits: {
  globalMax: env.RATE_LIMIT_GLOBAL_MAX,
  globalWindowMs: env.RATE_LIMIT_GLOBAL_WINDOW_MS,
```

```
    authMax: env.RATE_LIMIT_AUTH_MAX,
    authIpMax: env.RATE_LIMIT_AUTH_IP_MAX,
    authEmailMax: env.RATE_LIMIT_AUTH_EMAIL_MAX,
    authWindowMs: env.RATE_LIMIT_AUTH_WINDOW_MS,
    userMax: env.RATE_LIMIT_USER_MAX,
    userWindowMs: env.RATE_LIMIT_USER_WINDOW_MS,
    expensiveMax: env.RATE_LIMIT_EXPENSIVE_MAX,
    expensiveWindowMs: env.RATE_LIMIT_EXPENSIVE_WINDOW_MS,
    searchMax: env.RATE_LIMIT_SEARCH_MAX,
    searchWindowMs: env.RATE_LIMIT_SEARCH_WINDOW_MS,
    patronCardVerifyMax: env.RATE_LIMIT_PATRON_CARD_VERIFY_MAX,
    patronCardVerifyWindowMs:
env.RATE_LIMIT_PATRON_CARD_VERIFY_WINDOW_MS,
  },
  loginLockout: {
    maxAttempts: env.LOGIN_LOCKOUT_MAX_ATTEMPTS,
    durationS: env.LOGIN_LOCKOUT_DURATION_S,
    windowS: env.LOGIN_LOCKOUT_WINDOW_S,
  },
  trustProxyHops: env.TRUST_PROXY_HOPS !== undefined ?
env.TRUST_PROXY_HOPS : 1,
  socketIoRedisAdapterEnabled:
    env.SOCKET_IO_REDIS_ADAPTER_ENABLED !== undefined ?
env.SOCKET_IO_REDIS_ADAPTER_ENABLED : true,
  enableCron: env.ENABLE_CRON !== undefined ? env.ENABLE_CRON : true,
  timezoneAwareCronEnabled:
    env.TIMEZONE_AWARE_CRON_ENABLED !== undefined ?
env.TIMEZONE_AWARE_CRON_ENABLED : true,
```

```
  trustProxy: (() => {
    const val = env.TRUST_PROXY;
    if (val === true || val === 'true') return true;
    if (val === false || val === 'false' || val === undefined || val
=== null || val === '')
      return false;
    if (!isNaN(Number(val))) return Number(val);
    return val;
  })(),
  scannerApiKey: env.SCANNER_API_KEY,
  razorpayKeyId: env.RAZORPAY_KEY_ID,
  razorpayKeySecret: env.RAZORPAY_KEY_SECRET,
  healthCheckToken: env.HEALTH_CHECK_TOKEN || null,
  trustedProxies: env.TRUSTED_PROXIES ?
env.TRUSTED_PROXIES.split(',').map((p) => p.trim()) : [],
  additionalCorsOrigins: env.ADDITIONAL_CORS_ORIGINS
    ? env.ADDITIONAL_CORS_ORIGINS.split(',').map((o) => o.trim())
    : [],
  storage: {
    provider: env.STORAGE_PROVIDER || (env.BLOB_READ_WRITE_TOKEN ?
'vercel-blob' : 'local'),
    signingSecret: env.STORAGE_SIGNING_SECRET || env.JWT_SECRET ||
'bookbuddy-storage-secret',
    localDir: env.STORAGE_LOCAL_DIR || null,
    maxEbookSizeBytes: env.MAX_EBOOK_UPLOAD_SIZE_BYTES || 50 * 1024 *
1024,
    maxRosterSizeBytes: env.MAX_ROSTER_UPLOAD_SIZE_BYTES || 20 * 1024 *
1024,
  },
```

```
};


module.exports = config;
```

passport.js:

```
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;
const GitHubStrategy = require('passport-github2').Strategy;
const crypto = require('crypto');


const env = require('./env');
const User = require('../models/User');
const College = require('../models/College');
const logger = require('../utils/logger');


// Helper to resolve or create a default active college for OAuth users
const getDefaultCollege = async () => {
  let defaultCollege = await College.findOne({ isActive: true });
  if (!defaultCollege) {
    defaultCollege = await College.create({
      name: 'Main Campus',
      code: 'MAIN',
      isActive: true,
      status: 'active',
    });
```

```
  }

  return defaultCollege;

};



// Helper to resolve college using tenant context from verified OAuth
state

const resolveCollegeFromState = async (req) => {

  const state = req?.verifiedOAuthState;

  if (state?.collegeId) {

    const college = await College.findOne({ _id: state.collegeId,
isActive: true });

    if (college) return college;

  }

  if (state?.collegeSlug) {

    const college = await College.findOne({ slug: state.collegeSlug,
isActive: true });

    if (college) return college;

  }

  return getDefaultCollege();

};



//
------------------------------------------------------------------------
----

// Google OAuth Strategy

//
------------------------------------------------------------------------
----

const googleClientId = env.GOOGLE_CLIENT_ID ||
'placeholder_google_client_id';
```

```
const googleClientSecret = env.GOOGLE_CLIENT_SECRET ||
'placeholder_google_client_secret';

const googleCallbackUrl =

  env.GOOGLE_CALLBACK_URL ||
'http://localhost:5000/api/v1/auth/google/callback';



passport.use(

  new GoogleStrategy(

    {

      clientID: googleClientId,

      clientSecret: googleClientSecret,

      callbackURL: googleCallbackUrl,

      passReqToCallback: true,

    },

    async (req, accessToken, refreshToken, profile, done) => {

      try {

        const googleId = profile.id;

        const email =

          profile.emails && profile.emails[0] ?
profile.emails[0].value.toLowerCase() : null;

        const name = profile.displayName || profile.username || 'Google
User';

        const avatar = profile.photos && profile.photos[0] ?
profile.photos[0].value : '';



        if (!email) {

          return done(new Error('No email associated with Google
account'), null);

        }
```

```
    let user = await User.findOne({
      $or: [{ googleId }, { email }],
    });

    if (user) {
      let updated = false;
      if (!user.googleId) {
        user.googleId = googleId;
        updated = true;
      }
      if (avatar && !user.avatar) {
        user.avatar = avatar;
        updated = true;
      }
      if (!user.isEmailVerified) {
        user.isEmailVerified = true;
        updated = true;
      }
      if (updated) {
        await user.save();
      }
    } else {
      const targetCollege = await resolveCollegeFromState(req);
      const studentId = `G-${crypto.randomInt(100000, 999999)}`;

      user = await User.create({
```

```
            googleId,
            authProvider: 'google',
            studentId,
            name,
            email,
            avatar,
            collegeId: targetCollege._id,
            role: 'student',
            isEmailVerified: true,
            membershipStatus: 'active',
            status: 'active',
          });
        }

        return done(null, user);
      } catch (err) {
        logger.error(`Google Passport OAuth error: ${err.message}`);
        return done(err, null);
      }
    }
  )
);

//
-------------------------------------------------------------------------
----
// GitHub OAuth Strategy
```

```
//
------------------------------------------------------------------------
----

const githubClientId = env.GITHUB_CLIENT_ID ||
'placeholder_github_client_id';

const githubClientSecret = env.GITHUB_CLIENT_SECRET ||
'placeholder_github_client_secret';

const githubCallbackUrl =

  env.GITHUB_CALLBACK_URL ||
'http://localhost:5000/api/v1/auth/github/callback';



passport.use(

  new GitHubStrategy(

    {

      clientID: githubClientId,

      clientSecret: githubClientSecret,

      callbackURL: githubCallbackUrl,

      passReqToCallback: true,

    },

    async (req, accessToken, refreshToken, profile, done) => {

      try {

        const githubId = profile.id;

        const email =

          profile.emails && profile.emails[0]

            ? profile.emails[0].value.toLowerCase()

            : `${profile.username || githubId}@github.user`;

        const name = profile.displayName || profile.username || 'GitHub
User';
```

```
        const avatar = profile.photos && profile.photos[0] ?
profile.photos[0].value : '';


        let user = await User.findOne({
          $or: [{ githubId }, { email }],
        });


        if (user) {
          let updated = false;
          if (!user.githubId) {
            user.githubId = githubId;
            updated = true;
          }
          if (avatar && !user.avatar) {
            user.avatar = avatar;
            updated = true;
          }
          if (!user.isEmailVerified) {
            user.isEmailVerified = true;
            updated = true;
          }
          if (updated) {
            await user.save();
          }
        } else {
          const targetCollege = await resolveCollegeFromState(req);
          const studentId = `GH-${crypto.randomInt(100000, 999999)}`;
```

```
          user = await User.create({
            githubId,
            authProvider: 'github',
            studentId,
            name,
            email,
            avatar,
            collegeId: targetCollege._id,
            role: 'student',
            isEmailVerified: true,
            membershipStatus: 'active',
            status: 'active',
          });
        }

        return done(null, user);
      } catch (err) {
        logger.error(`GitHub Passport OAuth error: ${err.message}`);
        return done(err, null);
      }
    }
  )
);

module.exports = passport;
```

secretLoader.js:

```
const fs = require('fs');



/**

 * Loads secrets from a single JSON string or JSON file into the target environment (process.env).

 *

 * Evaluation Order:

 * 1. SECRETS_JSON (environment variable containing a JSON string)

 * 2. SECRETS_FILE_PATH (explicit file path to a JSON secrets file)

 * 3. /etc/bookbuddy/secrets.json (default production VM secrets path, if present)

 * 4. Fallback: No-op, preserves existing individual environment variables (Render / Vercel compatibility).

 *

 * Security:

 * - Prototype pollution protection (rejects __proto__, constructor, prototype)

 * - Secrets are NEVER logged to console or stdout

 *

 * @param {Object} options

 * @param {string} [options.secretsJson] - Optional override for SECRETS_JSON content

 * @param {string} [options.filePath] - Optional override for SECRETS_FILE_PATH

 * @param {string} [options.defaultPath] - Default file path (defaults to '/etc/bookbuddy/secrets.json')
```

```
 * @param {Object} [options.targetEnv] - Target environment object
(defaults to process.env)
 * @returns {{ loaded: boolean, source: string | null, count: number }}
 */
function loadSecrets(options = {}) {
  const targetEnv = options.targetEnv || process.env;
  const secretsJson =
    options.secretsJson !== undefined ? options.secretsJson :
targetEnv.SECRETS_JSON;
  const filePath = options.filePath !== undefined ? options.filePath :
targetEnv.SECRETS_FILE_PATH;
  const defaultPath =
    options.defaultPath !== undefined ? options.defaultPath :
'/etc/bookbuddy/secrets.json';

  let rawContent = null;
  let source = null;

  if (secretsJson && typeof secretsJson === 'string' &&
secretsJson.trim().length > 0) {
    rawContent = secretsJson.trim();
    source = 'SECRETS_JSON';
  } else if (filePath && typeof filePath === 'string' &&
filePath.trim().length > 0) {
    const resolvedPath = filePath.trim();
    if (!fs.existsSync(resolvedPath)) {
      throw new Error(`Explicit SECRETS_FILE_PATH not found:
${resolvedPath}`);
    }
```

```
    rawContent = fs.readFileSync(resolvedPath, 'utf8');
    source = `file:${resolvedPath}`;
  } else if (defaultPath && fs.existsSync(defaultPath)) {
    try {
      rawContent = fs.readFileSync(defaultPath, 'utf8');
      source = `file:${defaultPath}`;
    } catch (_err) {
      // Inaccessible default path is ignored to maintain fallback
compatibility
      return { loaded: false, source: null, count: 0 };
    }
  }

  if (!rawContent) {
    return { loaded: false, source: null, count: 0 };
  }

  let parsed;
  try {
    parsed = JSON.parse(rawContent);
  } catch (err) {
    throw new Error(`Failed to parse secrets JSON from ${source}:
${err.message}`);
  }

  if (!parsed || typeof parsed !== 'object' || Array.isArray(parsed)) {
    throw new Error(`Secrets JSON from ${source} must be a key-value
object`);
```

```
  }

  const forbiddenKeys = new Set(['__proto__', 'constructor',
'prototype']);
  let count = 0;

  for (const [key, value] of Object.entries(parsed)) {
    if (forbiddenKeys.has(key)) {
      continue;
    }
    if (value !== undefined && value !== null) {
      targetEnv[key] = typeof value === 'object' ?
JSON.stringify(value) : String(value);
      count += 1;
    }
  }

  return { loaded: true, source, count };
}

// Auto-execute on require
loadSecrets();

module.exports = {
  loadSecrets,
};
```

accessibility.js:

```
import { useEffect, useRef } from "react";
import { useTranslation } from "react-i18next";
import {
  Volume2,
  VolumeX,
  Play,
  Pause,
  Square,
  X,
  RotateCcw,
  Sliders,
  Type,
  SunMoon,
} from "lucide-react";
import { useAccessibilityStore } from "../../store/accessibilityStore";
import useAuthStore from "../../store/authStore";
import { useFeatureFlags } from "../../hooks/useFeatureFlags";
import { FEATURE_FLAGS } from "@bookbuddy/shared";


export function AccessibilityToolbar() {
  const { t } = useTranslation();
  const { user } = useAuthStore();
  const { isFeatureEnabled } = useFeatureFlags();
  const isLicensed =
isFeatureEnabled(FEATURE_FLAGS.ACCESSIBILITY_MODE);
```

```
const {
  currentUserId,
  setUserId,
  dyslexiaFont,
  highContrast,
  textToSpeech,
  ttsRate,
  isSpeaking,
  isPaused,
  isSpeechSupported,
  speechNotice,
  isToolbarOpen,
  toggleDyslexiaFont,
  toggleHighContrast,
  toggleTextToSpeech,
  setTtsRate,
  toggleToolbar,
  closeToolbar,
  resetPreferences,
  readCurrentPage,
  pauseSpeech,
  resumeSpeech,
  stopSpeech,
} = useAccessibilityStore();


const dialogRef = useRef(null);
```

```
  // Synchronize store userId when auth user changes
  useEffect(() => {
    const activeId = user?.id || user?._id || "guest";
    if (activeId !== currentUserId) {
      setUserId(activeId);
    }
  }, [user, currentUserId, setUserId]);

  // Keyboard navigation: Escape key closes toolbar
  useEffect(() => {
    const handleKeyDown = (e) => {
      if (e.key === "Escape" && isToolbarOpen) {
        closeToolbar();
      }
    };
    window.addEventListener("keydown", handleKeyDown);
    return () => window.removeEventListener("keydown", handleKeyDown);
  }, [isToolbarOpen, closeToolbar]);

  // If the college tenant has feature flag OFF, do not render toolbar
for users
  if (!isLicensed) {
    return null;
  }

  return (
```

```
    <aside
      role="complementary"
      aria-label={t("accessibility.toggleMenu", "Accessibility
Options")}
      className="fixed bottom-6 right-6 z-50 font-sans print:hidden"
    >
      {/* Floating Trigger Button */}
      <button
        type="button"
        onClick={toggleToolbar}
        aria-label={t("accessibility.toggleMenu", "Accessibility
Options")}
        aria-expanded={isToolbarOpen}
        aria-controls="accessibility-panel"
        className="flex items-center gap-2 px-3.5 py-3 rounded-full
bg-indigo-600 hover:bg-indigo-700 text-white shadow-xl
focus:outline-none focus:ring-4 focus:ring-indigo-300
dark:focus:ring-indigo-800 transition-all cursor-pointer font-medium
text-xs border border-white/20"
      >
        <Sliders className="w-4 h-4" aria-hidden="true" />
        <span className="hidden sm:inline">
          {t("accessibility.toggleMenu", "Accessibility")}
        </span>
      </button>

      {/* Accessibility Dialog / Drawer */}
      {isToolbarOpen && (
        <div
```

```
          id="accessibility-panel"

          ref={dialogRef}

          role="dialog"

          aria-modal="true"

          aria-labelledby="a11y-panel-title"

          className="absolute bottom-16 right-0 w-80 sm:w-96 bg-white
dark:bg-slate-900 border border-slate-200 dark:border-slate-800
rounded-2xl shadow-2xl p-5 text-slate-900 dark:text-slate-100
animate-in fade-in zoom-in-95 duration-150"

        >

          {/* Header */}

          <div className="flex items-center justify-between border-b
border-slate-100 dark:border-slate-800 pb-3 mb-4">

            <div className="flex items-center gap-2">

              <Sliders

                className="w-5 h-5 text-indigo-600
dark:text-indigo-400"

                aria-hidden="true"

              />

              <h2

                id="a11y-panel-title"

                className="text-sm font-bold tracking-tight"

              >

                {t("accessibility.title", "Accessibility Settings")}

              </h2>

            </div>

            <button

              type="button"

              onClick={closeToolbar}
```

```
              aria-label="Close accessibility options"
              className="p-1 rounded-lg text-slate-400
hover:text-slate-700 dark:hover:text-slate-200 hover:bg-slate-100
dark:hover:bg-slate-800 transition-colors"
            >
              <X className="w-4 h-4" aria-hidden="true" />
            </button>
          </div>

          <div className="space-y-4 text-xs">
            {/* Toggle 1: Dyslexia Font */}
            <div className="flex items-center justify-between p-3
rounded-xl bg-slate-50 dark:bg-slate-950 border border-slate-200/60
dark:border-slate-800/80">
              <div className="flex items-start gap-2.5 pr-2">
                <Type
                  className="w-4 h-4 text-indigo-500 mt-0.5
flex-shrink-0"
                  aria-hidden="true"
                />
                <div>
                  <div className="font-semibold text-slate-900
dark:text-slate-100">
                    {t("accessibility.dyslexiaFont", "Dyslexia-Friendly
Font")}
                  </div>
                  <p className="text-[11px] text-slate-500
dark:text-slate-400 leading-tight mt-0.5">
                    {t(
                      "accessibility.dyslexiaFontDesc",
```

```
                    "Uses OpenDyslexic typeface with enhanced letter
spacing",

                  )}

                </p>

              </div>

            </div>

            <button

              type="button"

              role="switch"

              aria-checked={dyslexiaFont}

              aria-label={t(

                "accessibility.dyslexiaFont",

                "Dyslexia-Friendly Font",

              )}

              onClick={toggleDyslexiaFont}

              className={`relative inline-flex h-6 w-11 flex-shrink-0
cursor-pointer rounded-full border-2 border-transparent
transition-colors duration-200 ease-in-out focus:outline-none
focus:ring-2 focus:ring-indigo-500 ${

                dyslexiaFont

                  ? "bg-indigo-600"

                  : "bg-slate-300 dark:bg-slate-700"

              }`}

            >

              <span

                className={`pointer-events-none inline-block h-5 w-5
transform rounded-full bg-white shadow-md ring-0 transition
duration-200 ease-in-out ${

                  dyslexiaFont ? "translate-x-5" : "translate-x-0"
```

```
                  }`}
                />
              </button>
            </div>

            {/* Toggle 2: High Contrast Mode */}
            <div className="flex items-center justify-between p-3
rounded-xl bg-slate-50 dark:bg-slate-950 border border-slate-200/60
dark:border-slate-800/80">
              <div className="flex items-start gap-2.5 pr-2">
                <SunMoon
                  className="w-4 h-4 text-amber-500 mt-0.5
flex-shrink-0"
                  aria-hidden="true"
                />
                <div>
                  <div className="font-semibold text-slate-900
dark:text-slate-100">
                    {t("accessibility.highContrast", "High Contrast
Mode")}
                  </div>
                  <p className="text-[11px] text-slate-500
dark:text-slate-400 leading-tight mt-0.5">
                    {t(
                      "accessibility.highContrastDesc",
                      "Increases contrast to WCAG AAA standards for
visual clarity",
                    )}
                  </p>
```

```
                </div>
              </div>
              <button
                type="button"
                role="switch"
                aria-checked={highContrast}
                aria-label={t(
                  "accessibility.highContrast",
                  "High Contrast Mode",
                )}
                onClick={toggleHighContrast}
                className={`relative inline-flex h-6 w-11 flex-shrink-0
cursor-pointer rounded-full border-2 border-transparent
transition-colors duration-200 ease-in-out focus:outline-none
focus:ring-2 focus:ring-indigo-500 ${
                  highContrast
                    ? "bg-indigo-600"
                    : "bg-slate-300 dark:bg-slate-700"
                }`}
              >
                <span
                  className={`pointer-events-none inline-block h-5 w-5
transform rounded-full bg-white shadow-md ring-0 transition
duration-200 ease-in-out ${
                    highContrast ? "translate-x-5" : "translate-x-0"
                  }`}
                />
              </button>
            </div>
```

```
            {/* Toggle 3: Text-to-Speech (TTS) Engine */}
            <div className="p-3 rounded-xl bg-slate-50
dark:bg-slate-950 border border-slate-200/60 dark:border-slate-800/80
space-y-3">
              <div className="flex items-center justify-between">
                <div className="flex items-start gap-2.5 pr-2">
                  {textToSpeech ? (
                    <Volume2
                      className="w-4 h-4 text-emerald-500 mt-0.5
flex-shrink-0"
                      aria-hidden="true"
                    />
                  ) : (
                    <VolumeX
                      className="w-4 h-4 text-slate-400 mt-0.5
flex-shrink-0"
                      aria-hidden="true"
                    />
                  )}
                  <div>
                    <div className="font-semibold text-slate-900
dark:text-slate-100">
                      {t("accessibility.textToSpeech", "Text to Speech
(TTS)")}
                    </div>
                    <p className="text-[11px] text-slate-500
dark:text-slate-400 leading-tight mt-0.5">
                      {t(
```

```
                        "accessibility.textToSpeechDesc",
                        "Reads on-screen content aloud using Web Speech
API",
                      )}
                    </p>
                  </div>
                </div>
                <button
                  type="button"
                  role="switch"
                  aria-checked={textToSpeech}
                  aria-label={t(
                    "accessibility.textToSpeech",
                    "Text to Speech (TTS)",
                  )}
                  onClick={toggleTextToSpeech}
                  className={`relative inline-flex h-6 w-11
flex-shrink-0 cursor-pointer rounded-full border-2 border-transparent
transition-colors duration-200 ease-in-out focus:outline-none
focus:ring-2 focus:ring-indigo-500 ${
                    textToSpeech
                      ? "bg-indigo-600"
                      : "bg-slate-300 dark:bg-slate-700"
                  }`}
                >
                  <span
                    className={`pointer-events-none inline-block h-5
w-5 transform rounded-full bg-white shadow-md ring-0 transition
duration-200 ease-in-out ${
```

```
                    textToSpeech ? "translate-x-5" : "translate-x-0"
                  }`}
                />
              </button>
            </div>

            {/* TTS Controls & Unsupported Fallback */}
            {textToSpeech && (
              <div className="pt-2 border-t border-slate-200
dark:border-slate-800 space-y-2.5">
                {!isSpeechSupported ? (
                  <div
                    role="alert"
                    className="p-2.5 rounded-lg bg-amber-50
dark:bg-amber-950/80 border border-amber-200 dark:border-amber-800
text-amber-800 dark:text-amber-200 text-[11px] leading-relaxed"
                  >
                    {t(
                      "accessibility.ttsUnsupported",
                      "Text-to-speech is not supported by your
current browser. Please use Google Chrome or a modern browser with Web
Speech API support.",
                    )}
                  </div>
                ) : (
                  <>
                    {/* Read / Pause / Stop Buttons */}
                    <div className="flex items-center gap-2">
                      {!isSpeaking ? (
```

```
                        <button
                          type="button"
                          onClick={readCurrentPage}
                          aria-label={t(
                            "accessibility.ttsPlay",
                            "Read Page Aloud",
                          )}
                          className="flex-1 flex items-center
justify-center gap-1.5 py-1.5 px-3 rounded-lg bg-emerald-600
hover:bg-emerald-700 text-white font-medium text-xs transition-colors
cursor-pointer"
                        >
                          <Play className="w-3.5 h-3.5"
aria-hidden="true" />
                          <span>
                            {t("accessibility.ttsPlay", "Read Page")}
                          </span>
                        </button>
                      ) : isPaused ? (
                        <button
                          type="button"
                          onClick={resumeSpeech}
                          aria-label={t(
                            "accessibility.ttsResume",
                            "Resume Speech",
                          )}
                          className="flex-1 flex items-center
justify-center gap-1.5 py-1.5 px-3 rounded-lg bg-indigo-600
```

```
hover:bg-indigo-700 text-white font-medium text-xs transition-colors
cursor-pointer"

                          >

                            <Play className="w-3.5 h-3.5"
aria-hidden="true" />

                            <span>

                              {t("accessibility.ttsResume", "Resume")}

                            </span>

                          </button>

                        ) : (

                          <button

                            type="button"

                            onClick={pauseSpeech}

                            aria-label={t(

                              "accessibility.ttsPause",

                              "Pause Speech",

                            )}

                            className="flex-1 flex items-center
justify-center gap-1.5 py-1.5 px-3 rounded-lg bg-amber-600
hover:bg-amber-700 text-white font-medium text-xs transition-colors
cursor-pointer"

                          >

                            <Pause className="w-3.5 h-3.5"
aria-hidden="true" />

                            <span>{t("accessibility.ttsPause",
"Pause")}</span>

                          </button>

                        )}
```

```
                        <button
                          type="button"
                          onClick={stopSpeech}
                          disabled={!isSpeaking && !isPaused}
                          aria-label={t(
                            "accessibility.ttsStop",
                            "Stop Reading",
                          )}
                          className="flex items-center justify-center
p-1.5 rounded-lg border border-slate-300 dark:border-slate-700
text-slate-600 dark:text-slate-300 hover:bg-slate-100
dark:hover:bg-slate-800 disabled:opacity-40 transition-colors
cursor-pointer"
                        >
                          <Square className="w-3.5 h-3.5"
aria-hidden="true" />
                        </button>
                      </div>

                      {/* Speed Rate Control */}
                      <div className="flex items-center justify-between
text-[11px] text-slate-500 dark:text-slate-400">
                        <span>
                          {t("accessibility.ttsSpeed", "Rate")}:
{ttsRate}x
                        </span>
                        <div className="flex items-center gap-1">
                          {[0.8, 1.0, 1.25, 1.5].map((speed) => (
                            <button
```

```
                                key={speed}
                                type="button"
                                onClick={() => setTtsRate(speed)}
                                className={`px-1.5 py-0.5 rounded
text-[10px] font-semibold transition-colors cursor-pointer ${
                                  ttsRate === speed
                                    ? "bg-indigo-600 text-white"
                                    : "bg-slate-200 dark:bg-slate-800
text-slate-700 dark:text-slate-300"
                                }`}
                              >
                                {speed}x
                              </button>
                            ))}
                          </div>
                        </div>

                        {/* Dynamic Speech Announcement Status */}
                        {speechNotice && (
                          <div
                            role="status"
                            aria-live="polite"
                            className="text-[10px] text-indigo-600
dark:text-indigo-400 font-medium italic"
                          >
                            {speechNotice}
                          </div>
                        )}
```

```
                    </>
                  )}
                </div>
              )}
            </div>

            {/* Reset Button */}
            <div className="pt-2 flex justify-between items-center
text-[11px]">
              <span className="text-slate-400 dark:text-slate-500">
                {t("accessibility.saved", "Preferences save
automatically")}
              </span>
              <button
                type="button"
                onClick={resetPreferences}
                className="flex items-center gap-1 text-slate-500
hover:text-slate-900 dark:hover:text-slate-200 transition-colors
cursor-pointer"
              >
                <RotateCcw className="w-3 h-3" aria-hidden="true" />
                <span>{t("accessibility.reset", "Reset")}</span>
              </button>
            </div>
          </div>
        </div>
      )}
    </aside>
```

```
  );

}
```

admin.js

4.2 Testing Approach

Chapter 5

Results and Discussions

1. Overview of Results

The project delivered a working, deployed, multi-tenant library platform covering physical circulation, a digital reader, lab booking, payments and gamification. It runs as four role-based portals on a shared backend.

Deliverable	Outcome achieved
Role-based portals	4 portals: Public Discovery, Student, College Admin (13 desks), Super Admin (10 consoles)

Functional modules	15 modules (see the Module Division chapter)
Data layer	73 Mongoose domain models in 7 clusters
REST API	121+ versioned endpoints under /api/v1, validated by central Zod contracts
Background automation	12 scheduled node-cron jobs with execution locks and run logging
Real-time layer	Socket.io with Redis adapter and six room types
Quality gates	10 automated CI invariant gates
Live deployment	Client on Vercel, API on Render, with /health and /version telemetry

2. Functional Results

2.1 Public / Visitor Portal

Visitors can search the OPAC catalog with debounced autocomplete and facet filters.

External catalogs (Google Books, Open Library, Project Gutenberg) work as fallbacks.

Open-access e-resources can be previewed in the browser without an account.

Wildcard subdomain and /c/: college Slug/ routing scope the portal to the active institution.

Guests can save bookmarks locally, and the home page loads from a single aggregated endpoint.

2.2 Student Portal

Students can borrow, renew (up to 2 times by default), place holds and see queue position.

Overdue fines can be paid online through Razor pay with signature verification.

The in-browser reader supports EPUB and PDF with HTTP 206 streaming, highlights, notes and cross-device position sync.

Offline reading works through Indexed DB and a service worker.

Lab seat booking uses a visual grid with waitlist promotion and no-show auto-release. ● Gamification features (streaks with freeze tokens, badges, leader board) are live.

Reviews, shelves, recommendations, the community feed and support tickets are available.

A digital patron card carries a Code128 barcode and QR code.

2.3 College Admin Portal

All 13 desks are implemented: Circulation, Patrons, Cataloguing, Inventory, Finances, Facilities, Digital Assets, Acquisitions, Feature Manager, Bulk Upload, Share Requests (ILL), Helpdesk and Analytics.

Each college can switch service modules on or off and set its own borrowing policy.

2.4 Super Admin Portal

Colleges and their first admin are provisioned in one atomic operation.

Admins can run the on boarding review queue, global moderation, user directory and immutable audit logs.

Telemetry, maintenance mode and guarded database restore are available.

MFA-gated impersonation shows a persistent warning banner.

3. Performance Results

Metric	Before	After	Improvement
Registration Latency	Approximately 1,200 ms	Approximately 150 ms	Approximately 8× faster
Primary Entry Bundle	Over 1.8 MB	Chunks under 350 KB	Approximately 80% reduction
Bulk Student Import	Synchronous upload caused HTTP 504 timeouts	Asynchronous HTTP 202 pipeline, 500-record chunks, and live progress tracking	Supports 5,000+ records per batch

How the gains were achieved:

Registration: bcrypt salt rounds were tuned to 10, and email and audit logging were moved to un-awaited background jobs.

Bundle: Vite manual chunks split React, router, TanStack, framer-motion, epubjs, pdfjs and recharts.

Bulk import: insertMany({ordered:false}) in chunks of 500, with an error-report CSV.

Targets enforced by the project's checks:

LCP under 2.5 s, CLS under 0.1, INP under 200 ms.

Gzipped landing bundle under 200 KB, checked by check-bundle-budget.js.

Sub-50 ms gate-scanner verification, limited to 60 requests per minute.

Skeleton-loader CLS under 0.05.

4. Security Results

Threat	**Mitigation verified**
Stolen refresh token	Rotation with reuse detection revokes the entire session family.
CSRF	_csrf cookie matched against X-CSRF-Token header on all mutating requests.
Cross-tenant leakage	collegeId filter injection, compound indexes and a static AST audit (Gate 3) over all 73 models and 53 controllers.
NoSQL injection / HTTP parameter pollution	express-mongo-sanitize and hpp.
Payment tampering / replay	HMAC-SHA256 verification and idempotent webhooks.
SSRF via cover or metadata fetch	Domain allowlists blocking loopback, link-local and RFC1918 ranges.
Privilege escalation	Canonical ROLES constants, last-super-admin protection, nested impersonation blocked.
Audit tampering	AuditLog immutable at the Mongoose layer.
CSV formula injection	Cells starting with =, +, - or @ are neutralised on export.

ReDoS / full scans	Regex-escaped search and a hard page-size cap of 100.

5. Testing and Quality Results

Test layer	Result reported
Backend Jest integration tests	80+ suites, 450+ tests on isolated MongoDB instances
Frontend Vitest	20 suites
Landing page verification	18-point checklist, each point backed by an automated test
Super Admin verification	22-module production checklist, concluding GO FOR PRODUCTION RELEASE
Concurrency	Gate 10 stress-tests parallel seat bookings, stock decrements and fine settlements

The 10 CI gates:

Gate	Invariant enforced
	No high or critical dependency CVEs
	No leaked secrets in files or diffs
	Tenant isolation on all models and controllers
	No N+1 queries inside loops
	Multi-document writes use runInTransaction
	Contended updates use atomicConditionalUpdate
	All routes use shared Zod contracts
	Lowercase identity and canonical roles
	No unbounded COLLSCAN queries
	Concurrency race invariants hold

6. Usability, Accessibility and Localisation Results

The layout was verified from 320 px to 1920 px, with no horizontal overflow and touch targets of at least 44 x 44 px.

WCAG 2.1 AA measures are in place: keyboard navigation, focus rings, landmarks and contrast of at least 4.5:1.

The UI is available in English, Hindi, Marathi and Spanish.

The navigation breakpoint moved from 1024 px to 1280 px, which removed wrapping and collisions. Carousel cards were resized to a 490 px height so shelf location and badges are no longer clipped.

Zero-scroll 100vh portals keep filters and toolbars visible.

7. Deployment Results

Client: Vercel edge CDN. API: Render. Production-style setup: GCP VM with Nginx and Cloudflare Full (Strict) TLS.

Hashed assets are cached for one year as immutable. index.html, sw.js and version.json are never cached, so releases reach users immediately.

Public catalog responses are edge-cached (max-age=300) and per-user data uses private, no-store.

verify-deployment.js smoke-tests the root page, book catalog, e-resource catalog, active colleges and cluster health.

A rollback runbook covers Vercel, Render, Nginx releases and MongoDB migrations.

8. Objectives Versus Outcomes

Objective	Result
Unify physical, digital and facility services	Achieved in a single platform
Serve many colleges on one cluster	Tenant isolation, subdomains and per-college features
Secure sessions and payments	Token-family revocation, CSRF, HMAC verification
Scale patron onboarding	Async 5,000+ record ingestion
Prevent double-booking	Atomic conditional updates and concurrency suite
Increase student engagement	Streaks, badges, leaderboards, reviews and recommendations
Production readiness	Live deployment, CI gates and runbooks

Chapter 6 Conclusion and Future Work

Chapter 7 References